arXiv:2509.11995v1 [math.OC] 15 Sep 2025

SSNCVX: A primal-dual semismooth Newton method for convex composite optimization problem

Zhanwang Deng* Tao Wei † Jirui Ma ‡ Zaiwen Wen §

September 16, 2025

Abstract

In this paper, we propose a uniform semismooth Newton-based algorithmic framework called SSNCVX for solving a broad class of convex composite optimization problems. By exploiting the augmented Lagrangian duality, we reformulate the original problem into a saddle point problem and characterize the optimality conditions via a semismooth system of nonlinear equations. The nonsmooth structure is handled internally without requiring problem specific transformation or introducing auxiliary variables. This design allows easy modifications to the model structure, such as adding linear, quadratic, or shift terms through simple interface-level updates. The proposed method features a single loop structure that simultaneously updates the primal and dual variables via a semismooth Newton step. Extensive numerical experiments on benchmark datasets show that SSNCVX outperforms state-of-the-art solvers in both robustness and efficiency across a wide range of problems.

Keywords: Convex composite optimization, augmented Lagrangian duality, semismooth Newton method.

1 Introduction

In this paper, we aim to develop an algorithmic framework for the following convex composite problem:

$$\begin{aligned} \min_{\boldsymbol{x}} \quad & p(\boldsymbol{x}) + f(\mathcal{B}(\boldsymbol{x})) + \langle \boldsymbol{c}, \boldsymbol{x} \rangle + \frac{1}{2} \langle \boldsymbol{x}, \mathcal{Q}(\boldsymbol{x}) \rangle, \\ \text{s.t.} \quad & \boldsymbol{x} \in \mathcal{P}_1, \quad \mathcal{A}(\boldsymbol{x}) \in \mathcal{P}_2, \end{aligned} \tag{1.1}$$

where $p(\boldsymbol{x})$ is a convex and nonsmooth function, $\mathcal{A} : \mathcal{X} \to \mathbb{R}^m, \mathcal{B} : \mathcal{X} \to \mathbb{R}^l$ are linear operators, $f : \mathbb{R}^l \to \mathbb{R}$ is a convex function, $\boldsymbol{c} \in \mathcal{X}$, $\mathcal{Q}$ is a positive semidefinite matrix or operator, $\mathcal{P}_1 = \{\boldsymbol{x} \in \mathcal{X} | \mathtt{l} \leq \boldsymbol{x} \leq \mathtt{u}\}$ and $\mathcal{P}_2 = \{\boldsymbol{x} \in \mathbb{R}^m | \mathtt{lb} \leq \boldsymbol{x} \leq \mathtt{ub}\}$. The choices of $p(\boldsymbol{x})$ provide flexibility to handle many types of problems. While the model (1.1) focuses on a single variable $\boldsymbol{x}$, it is indeed capable of solving the following more general problem with N blocks of variables with shifting terms $\boldsymbol{b}_{1,i}$ and $\boldsymbol{b}_{2,i}, (i = 1, \cdots, N)$:

$$\begin{aligned} \min_{\boldsymbol{x}_i} \quad & \sum_{i=1}^{N} p_i(\boldsymbol{x}_i - \boldsymbol{b}_{1,i}) + \sum_{i=1}^{N} f_i(\mathcal{B}_i(\boldsymbol{x}) - \boldsymbol{b}_{2,i}) + \sum_{i=1}^{N} \langle \boldsymbol{c}_i, \boldsymbol{x}_i \rangle + \sum_{i=1}^{N} \frac{1}{2} \langle \boldsymbol{x}_i, \mathcal{Q}_i(\boldsymbol{x}_i) \rangle, \\ \text{s.t.} \quad & \boldsymbol{x}_i \in \mathcal{P}_{1,i}, \quad \sum_{i=1}^{N} \mathcal{A}_i(\boldsymbol{x}_i) \in \mathcal{P}_{2,i}, \quad i = 1, \cdots, N, \end{aligned} \tag{1.2}$$

*Center of Machine learning, Peking University (email: dzw_opt2022@stu.pku.edu.cn).

†Center of Machine learning, Peking University (email: weit@pku.edu.cn)

‡Beijing International Center for Mathematical Research, Peking University (email: majirui@pku.edu.cn)

§Beijing International Center for Mathematical Research, Center for Machine Learning Research, Changsha Institute for Computing and Digital Economy, Peking University (email: wenzw@pku.edu.cn).

where $p_i, f_i, c_i, \mathcal{Q}_i, \mathcal{P}_{1,i}$, and $\mathcal{P}_{2,i}$ satisfy the same assumptions in (1.1). Models (1.1) and (1.2) have widespread applications in engineering, image processing, and machine learning, etc. We refer the readers to [8, 1, 46, 11, 7] for more concrete applications.

1.1 Related works

The first-order methods are popular for solving (1.1) due to the easy implementation and rapid convergence speed to a moderate accuracy point. For SDP and SDP+ problems, the alternating direction method of multipliers (ADMM), as implemented in SDPAD [44], has demonstrated considerable numerical efficiency. A convergent symmetric Gauss–Seidel based three-block ADMM method is developed in [39], which is capable of handling SDP problems with additional polyhedral set constraints. ABIP and ABIP+ [12] are new interior point methods for conic programming. ABIP uses a few steps of ADMM to approximately solve the subproblems that arise when applying a path-following barrier algorithm to the homogeneous self-dual embedding of the problem. SCS [34, 36] is an ADMM-based solver for convex quadratic cone programs implemented in C that applies ADMM to the homogeneous self-dual embedding of the problem, which yields infeasibility certificates when appropriate. TFOCS [6] and FOM [5] are solvers that aim to solve convex composite optimization problems using a class of first-order algorithms such as the Nesterov-type accelerated method.

The interior point method (IPM) is a classical approach for solving a subclass of (1.1), particularly for conic programming. There are well-designed open source solvers based on the interior point methods, such as SeDuMi [38] and SDPT3 [42]. For commercial solvers, MOSEK [2] is a high-performance optimization package specializing in large-scale convex problems (e.g., LP, QP, SOCP, SDP). Another state-of-the-art solver, Gurobi [35], excels in speed and scalability for complex optimization tasks, including LP, SOCP, and QP. Building on these solvers, CVX [18] is a MATLAB-based modeling framework for convex optimization, while its Python counterpart CVXPY [15] offers similar functionality. When addressing conic constraints in (1.1), the interior point methods rely on smooth barrier functions to ensure that the iterates lie within the cone. If direct methods are used to solve the linear equation, each iteration of IPMs requires factorizing the Schur complement matrix, which becomes increasingly costly in both computational and memory as the constraint dimension of the problem grows. Moreover, when iterative methods are used in this context, they often fail to exploit the sparse or low-rank structure of the solution. Furthermore, for general nonsmooth terms, the interior point methods cannot handle them directly. For instance, problems involving $\|\boldsymbol{x}\|_1$ are typically first reformulated as linear programs and then solved using interior-point methods [6, 9].

The semismooth Newton (SSN) methods are also effective for solving certain subclasses of problems in (1.1), such as Lasso [25, 47] and semidefinite programming (SDP) [27, 48]. One class of SSN methods integrates SSN into the augmented Lagrangian method (ALM) framework to solve subproblems of the primal variable, such as SDPNAL+ [40] for SDP with bound constraints and SSNAL [25] for Lasso problems. In addition, SSN can also be applied directly to solve a single nonlinear system derived from optimality conditions. A regularized semismooth Newton method is proposed [47] to solve two-block composite optimization problems such as Lasso and basis pursuit problems. Based on the equivalence of DRS iteration and ADMM, an efficient solver named SSNSDP for SDP is designed [27]. The idea is further extended to solving optimal transport problems [31]. However, their analysis of superlinear convergence relies on the BD regularity, which indicates that the solution is isolated. To alleviate this problem, based on the strict complementary and local error bound condition, the superlinear linear convergence of regularized SSN for composite optimization is proposed in [20]. Algorithms based on DRS or proximal gradient mapping can only handle two-block problems. To alleviate this problem, based on the saddle point problems induced from the augmented Lagrangian duality [13], an efficient method called

ALPDSN is designed for multi-block problems. It also demonstrates considerable performance on various SDP benchmarks [14]. A decomposition method called SDPDAL [43] is employed to handle SDP and QSDP with bound constraints, where the subproblem is solved using a semismooth Newton approach. Compared with the interior point methods, the semismooth Newton methods make use of the intrinsic sparse or low-rank structure efficiently, resulting in low memory requirements and low computational cost at each iteration. Therefore, developing a convex optimization framework specifically designed for multi-block practical applications is of theoretical and practical significance.

1.2 Contribution

We develop an SSN-based general-purpose optimization framework for solving the broad class of problems described in Model (1.1). The contributions of this paper are listed as follows.

- A practical model encompasses various optimization problems with nonsmooth terms or constraints (see Table 1 for details). By leveraging the AL duality, we transform the original problem (1.1) into a saddle point problem and formulate a semismooth system of nonlinear equations to characterize the optimality conditions. Unlike the interior point methods, our framework handles nonsmooth terms such as coupling conic constraints and simple norm constraints in standard form, without additional relaxation variables. Furthermore, it is more user-friendly, allowing for easy modifications to the optimization model, such as adding linear, quadratic, or shift terms. Instead of designing separate algorithms for each problem, the proposed framework requires only the selection of different functions and constraints, with updates made solely at the interface level.

- A unified algorithmic framework can handle complex multi-block semismooth systems. Unlike some SSN-based methods that rely on switching to first-order steps (e.g., fixed point iteration or ADMM) to ensure convergence, our approach retains second-order information at every iteration, ensuring faster and more robust convergence. Furthermore, we introduce a systematic approach for calculating generalized Jacobians, enabling efficient second-order updates for a broad class of nonsmooth functions. For certain complex non-smooth functions, we provide the detailed derivations of computationally efficient implementations. These effective computational approaches enable the practical utilization of both low-rank and sparse structures within the corresponding non-smooth functions.

- Comprehensive and promising numerical results. To rigorously evaluate the performance of SSNCVX, we conduct extensive experiments across a wide range of optimization problems, including Lasso, fused Lasso, SOCP, QP, and SPCA problems. SSNCVX demonstrates superior performance compared to state-of-the-art solvers on all these problems. These results not only validate SSNCVX as a highly efficient and reliable solver but also underscore its potential as a versatile tool for large-scale optimization tasks in related fields such as machine learning and signal processing.

1.3 Notation

For a linear operator $\mathcal{A}$, its adjoint operator is denoted by $\mathcal{A}^*$. For a proper convex function g, we define its domain as $\mathrm{dom}(g) := \{\boldsymbol{x} : g(\boldsymbol{x}) < \infty\}$. The Fenchel conjugate function of g is $g^*(\boldsymbol{z}) := \sup_{\boldsymbol{x}}\{\langle \boldsymbol{x}, \boldsymbol{z}\rangle - g(\boldsymbol{x})\}$ and the subdifferential is $\partial g(\boldsymbol{x}) := \{\boldsymbol{z} : \ g(\boldsymbol{y}) - g(\boldsymbol{x}) \geq \langle \boldsymbol{z}, \boldsymbol{y} - \boldsymbol{x}\rangle \,,\ \forall \boldsymbol{y}\}$. For a convex set $\mathcal{Q}$, we use the notation $\delta_{\mathcal{Q}}$ to denote the indicator function of the set $\mathcal{Q}$, which takes the value 0 on $\mathcal{Q}$ and $+\infty$ elsewhere. The relative interior of $\mathcal{Q}$ is denoted by $\mathrm{ri}(\mathcal{Q})$. For any proper closed convex function g, and constant $t > 0$, the proximal operator of g is defined by $\mathrm{prox}_{tg}(\boldsymbol{x}) = \arg\min_{\boldsymbol{y}}\{g(\boldsymbol{y}) + \frac{1}{2t}\|\boldsymbol{y} - \boldsymbol{x}\|^2\}$. The Moreau envelope function of g is defined as $e_t g(x) = \min_{\boldsymbol{y}}\{g(\boldsymbol{y}) + \frac{t}{2}\|\boldsymbol{y} - \boldsymbol{x}\|^2\}$. When $g = \delta_{\mathcal{C}}(\boldsymbol{x})$ is the indicator

function of a convex set $\mathcal{C}$, it holds that $\mathrm{prox}_{tg}(\boldsymbol{x}) = \Pi_{\mathcal{C}}(\boldsymbol{x})$, where $\Pi_{\mathcal{C}}$ denotes the projection onto the set $\mathcal{C}$.

1.4 Organization

The rest of this paper is organized as follows. A primal-dual semismooth Newton method based on the AL duality is introduced in Section 2. The properties of the proximal operator are introduced in Section 3. Extensive experiments on various problems are conducted in Section 4 and we conclude this paper in Section 5.

2 A primal-dual semismooth Newton method

In this section, we introduce a primal-dual semismooth Newton method to solve the original problem (1.1). We first transform (1.1) into a saddle point problem using the AL duality in Section 2.1. Subsequently, a monotone nonlinear system induced by the saddle point problem is presented. Such a nonlinear system is semismooth and equivalent to the Karush–Kuhn–Tucker (KKT) optimality condition of problem (1.1). We then introduce an SSN method to solve the nonlinear system in Section 2.2. The efficient calculation of the Jacboian matrix to solve the linear system is introduced in Section 2.3 and some implementation details of the algorithm are presented in Section 2.4.

2.1 An equivalent saddle point problem

The procedure of handing (1.1) is similar to that of [13]. However, as the problem being dealt with is more practical and complex, we provide the full algorithmic derivation below for both completeness and reader comprehension. The dual problem of (1.1) can be represented by

$$\begin{aligned} \min_{\boldsymbol{y},\boldsymbol{z},\boldsymbol{s},\boldsymbol{r},\boldsymbol{v}} \quad & \delta^*_{\mathcal{P}_2}(-\boldsymbol{y}) + f^*(-\boldsymbol{z}) + p^*(-\boldsymbol{s}) + \frac{1}{2}\langle \mathcal{Q}\boldsymbol{v}, \boldsymbol{v}\rangle + \delta^*_{\mathcal{P}_1}(-\boldsymbol{r}), \\ \text{s.t.} \quad & \mathcal{A}^*(\boldsymbol{y}) + \mathcal{B}^*\boldsymbol{z} + \boldsymbol{s} - \mathcal{Q}\boldsymbol{v} + \boldsymbol{r} = \boldsymbol{c}. \end{aligned} \tag{2.1}$$

Introducing the slack variables $\boldsymbol{o}, \boldsymbol{q}, \boldsymbol{t}$, the equivalent optimization problem is

$$\begin{aligned} \min_{\boldsymbol{y},\boldsymbol{z},\boldsymbol{s},\boldsymbol{r},\boldsymbol{v},\boldsymbol{o},\boldsymbol{q},\boldsymbol{t}} \quad & \delta^*_{\mathcal{P}_2}(-\boldsymbol{o}) + f^*(-\boldsymbol{q}) - \langle \boldsymbol{b}_1, \boldsymbol{s}\rangle + p^*(-\boldsymbol{s}) + \frac{1}{2}\langle \mathcal{Q}\boldsymbol{v}, \boldsymbol{v}\rangle + \delta^*_{\mathcal{P}_1}(-\boldsymbol{t}), \\ \text{s.t.} \quad & \mathcal{A}^*(\boldsymbol{y}) + \mathcal{B}^*\boldsymbol{z} + \boldsymbol{s} - \mathcal{Q}\boldsymbol{v} + \boldsymbol{r} = \boldsymbol{c}, \quad \boldsymbol{y} = \boldsymbol{o}, \quad \boldsymbol{z} = \boldsymbol{q}, \quad \boldsymbol{r} = \boldsymbol{t}. \end{aligned} \tag{2.2}$$

The augmented Lagrangian function of (2.2) is

$$\begin{aligned} \mathcal{L}_\sigma(\boldsymbol{y}, \boldsymbol{s}, \boldsymbol{z}, \boldsymbol{r}, \boldsymbol{v}, \boldsymbol{o}, \boldsymbol{q}, \boldsymbol{t}, \boldsymbol{x}_1, \boldsymbol{x}_2, \boldsymbol{x}_3, \boldsymbol{x}_4) &= \delta^*_{\mathcal{P}_2}(-\boldsymbol{o}) + f^*(-\boldsymbol{q}) + p^*(-\boldsymbol{s}) - \langle \boldsymbol{b}_1, \boldsymbol{s}\rangle + \frac{1}{2}\langle \mathcal{Q}(\boldsymbol{v}), \boldsymbol{v}\rangle \\ &+ \delta^*_{\mathcal{P}_1}(-\boldsymbol{t}) + \frac{\sigma}{2}\left(\|\boldsymbol{o} - \boldsymbol{y} + \frac{1}{\sigma}\boldsymbol{x}_1\|_{\mathrm{F}}^2 + \|\boldsymbol{q} - \boldsymbol{z} + \frac{1}{\sigma}\boldsymbol{x}_2\|_{\mathrm{F}}^2 + \|\boldsymbol{t} - \boldsymbol{r} + \frac{1}{\sigma}\boldsymbol{x}_3\|^2\right) \\ &+ \frac{\sigma}{2}(\|\mathcal{A}^*(\boldsymbol{y}) + \mathcal{B}^*\boldsymbol{z} + \boldsymbol{s} - \mathcal{Q}\boldsymbol{v} + \boldsymbol{r} - \boldsymbol{c} + \frac{1}{\sigma}\boldsymbol{x}_4\|_{\mathrm{F}}^2) - \frac{1}{2\sigma}\sum_{i=1}^{4}\|\boldsymbol{x}_i\|^2. \end{aligned}$$

Minimizing $\mathcal{L}_\sigma$ with respect to the variables $\boldsymbol{o}, \boldsymbol{q}, \boldsymbol{s}, \boldsymbol{t}$ yields

$$\begin{aligned} & \boldsymbol{o} = -\mathrm{prox}_{\delta^*_{\mathcal{P}_2}/\sigma}(\boldsymbol{x}_1/\sigma - \boldsymbol{y}), \quad \boldsymbol{q} = -\mathrm{prox}_{f^*/\sigma}(\boldsymbol{x}_2/\sigma - \boldsymbol{z}), \\ & \boldsymbol{s} = -\mathrm{prox}_{p^*/\sigma}(\mathcal{A}^*(\boldsymbol{y}) + \mathcal{B}^*\boldsymbol{z} - \mathcal{Q}\boldsymbol{v} + \boldsymbol{r} - \boldsymbol{c} + \frac{1}{\sigma}\boldsymbol{x}_4), \quad \boldsymbol{t} = -\mathrm{prox}_{\delta^*_{\mathcal{P}_1}/\sigma}(\boldsymbol{x}_3/\sigma - \boldsymbol{r}). \end{aligned} \tag{2.3}$$

Problem	**Objective Function**	**Constraints**	**Function block**
	$\underbrace{\langle \boldsymbol{c}, \boldsymbol{x} \rangle}_{\text{(I)}} + \underbrace{\frac{1}{2} \langle \boldsymbol{x}, \mathcal{Q}(\boldsymbol{x}) \rangle}_{\text{(II)}} + \underbrace{f(\mathcal{B}(\boldsymbol{x}))}_{\text{(III)}} + p(\boldsymbol{x})$	$\underbrace{\boldsymbol{x} \in \mathcal{P}_1}_{\text{(IV)}}, \underbrace{\mathcal{A}(\boldsymbol{x}) \in \mathcal{P}_2}_{\text{(V)}}.$	(I) (II) (III) (IV) (V)
LP	$\langle \boldsymbol{c}, \boldsymbol{x} \rangle$	$\mathcal{A}(\boldsymbol{x}) = \boldsymbol{b}, \boldsymbol{x} \geq 0$	(I)(V)
SOCP	$\langle \boldsymbol{c}, \boldsymbol{x} \rangle$	$\mathcal{A}(\boldsymbol{x}) = \boldsymbol{b}, \boldsymbol{x} \in \mathcal{Q}^n$	(I)(V)
SDP	$\langle \boldsymbol{C}, \boldsymbol{X} \rangle$	$\mathcal{A}(\boldsymbol{X}) = \boldsymbol{b}, \boldsymbol{X} \succeq 0$	(I)(V)
SDP with box constraints	$\langle \boldsymbol{C}, \boldsymbol{X} \rangle$	$\mathcal{A}(\boldsymbol{X}) = \boldsymbol{b}, \boldsymbol{x} \in \mathcal{P}_1, \boldsymbol{X} \succeq 0$	(I)(IV)(V)
QP	$\langle \boldsymbol{x}, \mathcal{Q}(\boldsymbol{x}) \rangle + \langle \boldsymbol{x}, \boldsymbol{c} \rangle$	$\mathtt{l} \leq \boldsymbol{x} \leq \mathtt{u}, \mathcal{A}(\boldsymbol{x}) = \boldsymbol{b}$	(I)(II)(IV)(V)
QP with ℓ_1 norm	$\langle \boldsymbol{x}, \mathcal{Q}(\boldsymbol{x}) \rangle + \lambda \lVert \boldsymbol{x} \rVert_1$	$\mathtt{l} \leq \boldsymbol{x} \leq \mathtt{u}, \mathcal{A}(\boldsymbol{x}) = \boldsymbol{b}$	(I)(II)(III)(V)
Lasso	$\frac{1}{2} \lVert \mathcal{B}(\boldsymbol{x}) - \boldsymbol{b} \rVert^2 + \lambda \lVert \boldsymbol{x} \rVert_1$	-	(III)
Fused Lasso	$\frac{1}{2} \lVert \mathcal{B}(\boldsymbol{x}) - \boldsymbol{b} \rVert^2 + \lambda_1 \lVert \boldsymbol{x} \rVert_1 + \lambda_2 \lVert D\boldsymbol{x} \rVert_1$	-	(III)
Group Lasso	$\frac{1}{2} \lVert \mathcal{B}(\boldsymbol{x}) - \boldsymbol{b} \rVert^2 + \lambda \lVert \boldsymbol{x} \rVert_2$	-	(III)
Top-k Lasso	$\frac{1}{2} \lVert \mathcal{B}(\boldsymbol{x}) - \boldsymbol{b} \rVert^2 + \lambda \sum_{i=1}^{k} \boldsymbol{x}_{[i]}$	-	(III)
Low-rank matrix recovery	$\lVert \mathcal{B}(\boldsymbol{X}) - \boldsymbol{B} \rVert_{\mathrm{F}}^2 + \lambda \lVert \boldsymbol{X} \rVert_*$	-	(III)
Sparse covariance matrix estimation	$-\log(\det(\boldsymbol{X})) + \mathrm{Tr}(\boldsymbol{X}\boldsymbol{S}) + \lambda \lVert \boldsymbol{X} \rVert_1$	-	(I)(III)
Sparse PCA	$-\langle \boldsymbol{L}, \boldsymbol{x} \rangle + \lambda \lVert \boldsymbol{x} \rVert_1$	$\mathrm{Tr}(\boldsymbol{x}) = 1, \boldsymbol{x} \succeq 0$	(III)
Basis pursuit	$\lVert \boldsymbol{x} \rVert_1$	$\mathcal{A}(\boldsymbol{x}) = \boldsymbol{b}$	(V)
Robust PCA	$\lVert \boldsymbol{x}_1 \rVert_* + \lambda \lVert \boldsymbol{x}_2 \rVert_1$	$\boldsymbol{x}_1 + \boldsymbol{x}_2 = \boldsymbol{D}$	(III)(V)

Table 1: Examples that Model (1.1) is able to solve.

Let $\boldsymbol{w} = (\boldsymbol{y}, \boldsymbol{z}, \boldsymbol{r}, \boldsymbol{v}, \boldsymbol{x}_1, \boldsymbol{x}_2, \boldsymbol{x}_3, \boldsymbol{x}_4)$. Then the modified augmented Lagrangian function is:

$$\begin{aligned}\Phi_\sigma(\boldsymbol{w}) = & \underbrace{p^*(\mathrm{prox}_{p^*/\sigma}(\boldsymbol{x}_4/\sigma - \mathcal{A}^*(\boldsymbol{y}) - \mathcal{B}^*\boldsymbol{z} - \mathcal{Q}\boldsymbol{v} - \boldsymbol{r} + \boldsymbol{c})) + \frac{1}{2\sigma}\|\mathrm{prox}_{\sigma p}(\boldsymbol{x}_4 + \sigma(\mathcal{A}^*(\boldsymbol{y}) + \mathcal{B}^*\boldsymbol{z} - \mathcal{Q}\boldsymbol{v} + \boldsymbol{r} - \boldsymbol{c}))\|^2}_{\text{Moreau envelope } p^*} \\ & + \underbrace{\delta^*_{\mathcal{P}_1}(\mathrm{prox}_{\delta^*_{\mathcal{P}_1}}(\boldsymbol{x}_3/\sigma - \boldsymbol{t})) + \frac{1}{2\sigma}\|\Pi_{\mathcal{P}_1}(\boldsymbol{x}_3 - \sigma\boldsymbol{r})\|^2}_{\text{Moreau envelope } \delta^*_{\mathcal{P}_1}} + \underbrace{\delta^*_{\mathcal{P}_2}(\mathrm{prox}_{\delta^*_{\mathcal{P}_2}/\sigma}(\boldsymbol{x}_1/\sigma - \boldsymbol{y})) + \frac{1}{2\sigma}\|\Pi_{\mathcal{P}_2}(\boldsymbol{x}_1 - \sigma\boldsymbol{y})\|^2}_{\text{Moreau envelope } \delta^*_{\mathcal{P}_2}} \\ & + \underbrace{f^*(\mathrm{prox}_{f^*/\sigma}(\boldsymbol{x}_2/\sigma - \boldsymbol{z})) + \frac{1}{2\sigma}\|\mathrm{prox}_{\sigma f}(\boldsymbol{x}_2 - \sigma\boldsymbol{z})\|^2}_{\text{Moreau envelope } f^*} + \frac{1}{2}\langle \mathcal{Q}\boldsymbol{v}, \boldsymbol{v}\rangle - \frac{1}{2\sigma}\sum_{i=1}^{4}\|\boldsymbol{x}_i\|^2.\end{aligned} \tag{2.4}$$

Henceforth, the differentiable saddle point problem is

$$\min_{\boldsymbol{y},\boldsymbol{z},\boldsymbol{r},\boldsymbol{v}} \max_{\boldsymbol{x}_1,\boldsymbol{x}_2,\boldsymbol{x}_3,\boldsymbol{x}_4} \Phi(\boldsymbol{y}, \boldsymbol{z}, \boldsymbol{r}, \boldsymbol{v}; \boldsymbol{x}_1, \boldsymbol{x}_2, \boldsymbol{x}_3, \boldsymbol{x}_4). \tag{2.5}$$

In the subsequent analysis, we make the following assumption.

Assumption 1 (Slater's condition)**.** The dual problem (2.2) has an optimal solution $\boldsymbol{y}_*, \boldsymbol{z}_*, \boldsymbol{s}_*, \boldsymbol{r}_*, \boldsymbol{v}_*$. Furthermore, Slater's condition holds for the dual problem (2.1), i.e., there exists $-\boldsymbol{y} \in \mathrm{ri}(\mathrm{dom}(\delta^*_{\mathcal{P}_2}))$, $-\boldsymbol{s} \in \mathrm{ri}(\mathrm{dom}(p^*))$, $-\boldsymbol{r} \in \mathrm{dom}(\delta^*_{\mathcal{P}_1})$ and $-\boldsymbol{z} \in \mathrm{ri}(\mathrm{dom}(f^*))$ such that $\mathcal{A}^*(\boldsymbol{y}) + \mathcal{B}^*\boldsymbol{z} + \boldsymbol{s} - \mathcal{Q}\boldsymbol{v} + \boldsymbol{r} = \boldsymbol{c}$.

Based on Slater's condition, the saddle point problem satisfies the strong AL duality.

Lemma 1 (Strong duality [13])**.** *Suppose Assumption 1 holds. Given any $\sigma > 0$, the strong duality holds for* (2.5)*, i.e.,*

$$\min_{\boldsymbol{y},\boldsymbol{z},\boldsymbol{r},\boldsymbol{v}} \max_{\boldsymbol{x}_1,\boldsymbol{x}_2,\boldsymbol{x}_3,\boldsymbol{x}_4} \Phi(\boldsymbol{y}, \boldsymbol{z}, \boldsymbol{r}, \boldsymbol{v}; \boldsymbol{x}_1, \boldsymbol{x}_2, \boldsymbol{x}_3, \boldsymbol{x}_4) = \max_{\boldsymbol{x}_1,\boldsymbol{x}_2,\boldsymbol{x}_3,\boldsymbol{x}_4} \min_{\boldsymbol{y},\boldsymbol{z},\boldsymbol{r},\boldsymbol{v}} \Phi(\boldsymbol{y}, \boldsymbol{z}, \boldsymbol{r}, \boldsymbol{v}; \boldsymbol{x}_1, \boldsymbol{x}_2, \boldsymbol{x}_3, \boldsymbol{x}_4). \tag{2.6}$$

where both sides of (2.6) *are equivalent to problem* (1.1)*.*

2.2 A semismooth Newton method with global convergence

It follows from the Moreau envelope theorem [4] that $e_\sigma f^*$, $e_\sigma p^*$, $e_\sigma \delta^*_{\mathcal{Q}}$, and $e_\sigma \delta^*_{\mathcal{K}}$ are continuously differentiable, which implies that Φ is also continuously differentiable. Hence, the gradient of the saddle point problem can be represented by

$$\begin{aligned}
\nabla_{\boldsymbol{y}}\Phi_\sigma(\boldsymbol{w}) &= \mathcal{A}\mathrm{prox}_{\sigma p}(\boldsymbol{x}_4 + \sigma(\mathcal{A}^*(\boldsymbol{y}) + \mathcal{B}^*\boldsymbol{z} - \mathcal{Q}\boldsymbol{v} + \boldsymbol{r} - \boldsymbol{c})) - \Pi_{\mathcal{P}_2}(\boldsymbol{x}_1 - \sigma\boldsymbol{y}), \\
\nabla_{\boldsymbol{z}}\Phi_\sigma(\boldsymbol{w}) &= \mathcal{B}\mathrm{prox}_{\sigma p}(\boldsymbol{x}_4 + \sigma(\mathcal{A}^*(\boldsymbol{y}) + \mathcal{B}^*\boldsymbol{z} - \mathcal{Q}\boldsymbol{v} + \boldsymbol{r} - \boldsymbol{c})) - \mathrm{prox}_{\sigma f}(\boldsymbol{x}_2 - \sigma\boldsymbol{z}), \\
\nabla_{\boldsymbol{r}}\Phi_\sigma(\boldsymbol{w}) &= \mathrm{prox}_{\sigma p}(\boldsymbol{x}_4 + \sigma(\mathcal{A}^*(\boldsymbol{y}) + \mathcal{B}^*\boldsymbol{z} - \mathcal{Q}\boldsymbol{v} + \boldsymbol{r} - \boldsymbol{c})) - \Pi_{\mathcal{P}_1}(\boldsymbol{x}_3 - \sigma\boldsymbol{r}), \\
\nabla_{\boldsymbol{v}}\Phi_\sigma(\boldsymbol{w}) &= -\mathcal{Q}\mathrm{prox}_{\sigma p}(\boldsymbol{x}_4 + \sigma(\mathcal{A}^*(\boldsymbol{y}) + \mathcal{B}^*\boldsymbol{z} - \mathcal{Q}\boldsymbol{v} + \boldsymbol{r} - \boldsymbol{c})) + \mathcal{Q}\boldsymbol{v}, \\
\nabla_{\boldsymbol{x}_1}\Phi_\sigma(\boldsymbol{w}) &= \frac{1}{\sigma}\Pi_{\mathcal{P}_2}(\boldsymbol{x}_1 - \sigma\boldsymbol{y}) - \frac{1}{\sigma}\boldsymbol{x}_1, \\
\nabla_{\boldsymbol{x}_2}\Phi_\sigma(\boldsymbol{w}) &= \frac{1}{\sigma}\mathrm{prox}_{\sigma f}(\boldsymbol{x}_2 - \sigma\boldsymbol{z}) - \frac{1}{\sigma}\boldsymbol{x}_2, \\
\nabla_{\boldsymbol{x}_3}\Phi_\sigma(\boldsymbol{w}) &= \frac{1}{\sigma}\Pi_{\mathcal{P}_1}(\boldsymbol{x}_3 - \sigma\boldsymbol{r}) - \frac{1}{\sigma}\boldsymbol{x}_3, \\
\nabla_{\boldsymbol{x}_4}\Phi_\sigma(\boldsymbol{w}) &= \frac{1}{\sigma}\mathrm{prox}_{\sigma p}(\boldsymbol{x}_4 + \sigma(\mathcal{A}^*(\boldsymbol{y}) + \mathcal{B}^*\boldsymbol{z} - \mathcal{Q}\boldsymbol{v} + \boldsymbol{r} - \boldsymbol{c})) - \frac{1}{\sigma}\boldsymbol{x}_4.
\end{aligned} \tag{2.7}$$

We note that if f^* is differentiable, $\boldsymbol{x}_2$ does not exist and the corresponding gradient is $\nabla_{\boldsymbol{z}}\Phi_\sigma(\boldsymbol{w}) = \mathcal{B}\text{prox}_{\sigma p}(\boldsymbol{x}_4 + \sigma(\mathcal{A}^*(\boldsymbol{y}) + \mathcal{B}^*\boldsymbol{z} - \mathcal{Q}\boldsymbol{v} + \boldsymbol{r} - \boldsymbol{c})) - \nabla f^*(-\boldsymbol{z})$.

The nonlinear operator $F(\boldsymbol{w})$ is defined as

$$F(\boldsymbol{w}) = \left(\nabla_{\boldsymbol{y}}\Phi(\boldsymbol{w}); \nabla_{\boldsymbol{z}}\Phi(\boldsymbol{w}); \nabla_{\boldsymbol{r}}\Phi(\boldsymbol{w}); \nabla_{\boldsymbol{v}}\Phi(\boldsymbol{w}); -\nabla_{\boldsymbol{x}_1}\Phi(\boldsymbol{w}); -\nabla_{\boldsymbol{x}_2}\Phi(\boldsymbol{w}); -\nabla_{\boldsymbol{x}_3}\Phi(\boldsymbol{w}); -\nabla_{\boldsymbol{x}_4}\Phi(\boldsymbol{w})\right). \tag{2.8}$$

It is shown in [13, Lemma 3.1] that $\boldsymbol{w}_*$ is a solution of the saddle point problem (2.5) if and only if it satisfies $F(\boldsymbol{w}_*) = 0$. Hence, the saddle point problem can be transformed into solving the following nonlinear equations:

$$F(\boldsymbol{w}) = 0. \tag{2.9}$$

Definition 1. Let F be a locally Lipschitz continuous mapping. Denote by D_F the set of differentiable points of F. The B-Jacobian of F at $\boldsymbol{x}$ is defined by

$$\partial_B F(\boldsymbol{w}) := \left\{ \lim_{k\to\infty} J(\boldsymbol{w}^k) \,|\, \boldsymbol{w}^k \in D_F, \boldsymbol{w}^k \to \boldsymbol{w} \right\},$$

where $J(\boldsymbol{w})$ denotes the Jacobian of F at $\boldsymbol{w} \in D_F$. The set $\partial F(\boldsymbol{w}) = co(\partial_B F(\boldsymbol{w}))$ is called the Clarke subdifferential, where co denotes the convex hull.

F is semismooth at $\boldsymbol{w}$ if F is directionally differentiable at $\boldsymbol{w}$ and for any $\boldsymbol{d}$, $J \in \partial F(\boldsymbol{w}+\boldsymbol{d})$, it holds that $\|F(\boldsymbol{w}+\boldsymbol{d}) - F(\boldsymbol{w}) - J\boldsymbol{d}\| = o(\|\boldsymbol{d}\|), \quad \boldsymbol{d} \to 0$. F is said to be strongly semismooth at $\boldsymbol{w}$ if F is directionally differentiable at $\boldsymbol{w}$ and $\|F(\boldsymbol{w}+\boldsymbol{d}) - F(\boldsymbol{w}) - J\boldsymbol{d}\| = O(\|\boldsymbol{d}\|^2), \quad \boldsymbol{d} \to 0$. We say F is semismooth (strongly semismooth) if F is semismooth (strongly semismooth) for any $\boldsymbol{w}$ [32].

Note that for a convex function h, its proximal operator prox_{th} is Lipschitz continuous. Then, by Definition 1, we define the following sets:

$$\begin{aligned} &D_{\Pi_1} := \partial\Pi_{\mathcal{P}_1}(\boldsymbol{x}_3 - \sigma\boldsymbol{r}),\ D_{\Pi_2} = \partial\Pi_{\mathcal{P}_2}(\boldsymbol{x}_1 - \sigma\boldsymbol{y}),\ D_f := \partial\text{prox}_{\sigma f}(\boldsymbol{x}_2 - \sigma\boldsymbol{z}), \\ &D_p := \partial\text{prox}_{\sigma p}(\boldsymbol{x}_4 + \sigma(\mathcal{A}^*(\boldsymbol{y}) + \mathcal{B}^*\boldsymbol{z} - \mathcal{Q}\boldsymbol{v} + \boldsymbol{r} - \boldsymbol{c})). \end{aligned} \tag{2.10}$$

Hence, the corresponding generalized Jacobian can be represented by

$$\hat{\partial} F(\boldsymbol{w}) := \left\{ \begin{pmatrix} \mathcal{H}_{\mathbf{11}} & \mathcal{H}_{\mathbf{12}} \\ -\mathcal{H}_{\mathbf{12}}^\top & \mathcal{H}_{\mathbf{22}} \end{pmatrix} \right\}, \tag{2.11}$$

where

$$\begin{aligned} \mathcal{H}_{\mathbf{11}} &= \sigma\left(\mathcal{A}, \mathcal{B}, \mathcal{I}, -\mathcal{Q}\right)^{\mathrm{T}} D_p \left(\mathcal{A}, \mathcal{B}, \mathcal{I}, -\mathcal{Q}\right) + \sigma\text{blkdiag}(D_{\Pi_1}, D_f, D_{\Pi_2}, \mathcal{Q}), \\ \mathcal{H}_{\mathbf{12}} &= \left[\left(-\text{blkdiag}\left([D_{\Pi_1}, D_f, D_{\Pi_2}]\right); \mathbf{0}\right), \left(\mathcal{A}, \mathcal{B}, \mathcal{I}, -\mathcal{Q}\right)^{\mathrm{T}} D_p\right], \\ \mathcal{H}_{\mathbf{22}} &= \text{blkdiag}\left\{\frac{1}{\sigma}(\mathcal{I} - D_{\Pi_1}), \frac{1}{\sigma}(\mathcal{I} - D_h), \frac{1}{\sigma}(\mathcal{I} - D_{\Pi_2}), \frac{1}{\sigma}(\mathcal{I} - D_p)\right\}. \end{aligned} \tag{2.12}$$

It follows from [19] and the definition of $\hat{\partial}F$ that $\hat{\partial}F(\boldsymbol{w})[\boldsymbol{d}] = \partial F(\boldsymbol{w})[\boldsymbol{d}]$ for any $\boldsymbol{d}$. Hence, $\hat{\partial}F(\boldsymbol{w})$ is valid to construct a Newton equation to solve $F(\boldsymbol{w}) = 0$.

We next present the semismooth Newton method to solve (2.9). First, an element of the Clarke's generalized Jacobian is taken and defined by (2.11) as $J^k \in \hat{\partial}F(\boldsymbol{w}^k)$. Given $\tau_{k,i}$, we compute the semismooth Newton direction $\boldsymbol{d}^{k,i}$ as the solution of the following linear system

$$(J^k + \tau_{k,i}\mathcal{I})\boldsymbol{d}^{k,i} = -F(\boldsymbol{w}^k) + \boldsymbol{\varepsilon}^k, \tag{2.13}$$

where $\boldsymbol{\varepsilon}^k$ is the residual term to measure the inexactness of the equation. We require that there exists a constant $C_{\boldsymbol{\varepsilon}} > 0$ such that $\|\boldsymbol{\varepsilon}^k\| \le C_{\boldsymbol{\varepsilon}} k^{-\beta}$, $\beta \in (1/3, 1]$. The shift term $\tau_{k,i}\mathcal{I}$ is added to guarantee the existence and uniqueness of $\boldsymbol{d}^{k,i}$ and the trial step is defined by

$$\bar{\boldsymbol{w}}^{k,i} = \boldsymbol{w}^k + \boldsymbol{d}^{k,i}. \tag{2.14}$$

Next, we present a globalization scheme to ensure convergence only using regularized semismooth Newton steps. The main idea is to find a suitable $\tau_{k,i}$. It uses both line search on the shift parameter $\tau_{k,i}$ and the nonmonotone decrease on the residuals $F(\boldsymbol{w}^k)$. Specifically, for an integer $\zeta \geq 1$, $\nu \in (0,1), \kappa > 1, \gamma > 1, i_{\max} > 0, i = 0, \cdots, i_{\max}$, we aim to find the smallest i such that $\tau_{k,i} = \kappa\gamma^i \|F(\boldsymbol{w}^k)\|$ and the nonmonotone decrease condition

$$\|F(\bar{\boldsymbol{w}}^{k,i})\| \leq \nu \max_{\max(1,k-\zeta+1)\leq j\leq k} \|F(\boldsymbol{w}^j)\| + \varsigma_k \tag{2.15}$$

holds, where $\{\varsigma_i\}$ is a nonnegative sequence such that $\sum_{i=1}^{\infty} \varsigma_i^2 < \infty$. The iterative update $\boldsymbol{w}^{k+1} = \bar{\boldsymbol{w}}^{k,i}$ is performed if condition (2.15) holds. Otherwise, if (2.15) does not hold for $i > i_{\max}$, we choose $\tau_{k,i}$ such that

$$\tau_{k,i} \geq ck^{\beta}, \tag{2.16}$$

where $c > 0$ is a given constant and then we set $\boldsymbol{w}^{k+1} = \bar{\boldsymbol{w}}^{k,i}$.

Condition (2.15) assesses whether the residuals exhibit a nonmonotone sufficient descent property, which allows for temporary increases in residual values $\|F(\boldsymbol{w}^k)\|$. The parameters ζ and ν govern the number of previous points referenced in this evaluation, where larger values of ζ and ν lead to more lenient acceptance criteria for the semismooth Newton step. If (2.15) is not satisfied, the regularization parameter $\tau_{k,i}$ is adjusted according to (2.16), ensuring a monotonic decrease in the residual sequence $\{F(\boldsymbol{w}^k)\}$ through an implicit mechanism which combines a regularized semismooth Newton step. The nonmonotone strategy provides flexibility by imposing a relatively relaxed condition, which results in the acceptance condition (2.15) with the initial $\tau_{k,0}$ being satisfied in nearly all iterations, as empirically validated by our numerical experiments. The complete procedure is summarized in Algorithm 1.

Algorithm 1 A semismooth Newton method for solving (2.9).

Require: The constants $\gamma > 1$, $\nu \in (0,1)$, $\beta \in (1/2, 1]$, $\kappa > 0$, an integer $\zeta \geq 1$, and an initial point $\boldsymbol{w}^0$, set $k = 0$.

```
while stopping condition not met do
    Compute F(w^k) and choose one J(w^k) ∈ ∂̂F(w^k).
    Find the smallest i ≥ 0 such that w̄^{k,i} defined in (2.14) satisfies (2.15) or τ_{k,i} satisfy (2.16).
    Set w^{k+1} = w̄^{k,i}.
    Set k = k + 1.
end while
```

We have the following global convergence analysis of Algorithm 1 [13, Theorem 1].

Theorem 1. *Suppose that Assumption 1 holds. Let $\{\boldsymbol{w}^k\}$ be the sequence generated by Algorithm 1. The residual $F(\boldsymbol{w}^k)$ converges to 0, i.e.,*

$$\lim_{k\to\infty} F(\boldsymbol{w}^k) = 0. \tag{2.17}$$

For local convergence, we first introduce the definition of partial smoothness [24].

Definition 2 (C^p-partial smoothness)**.** Consider a proper closed function $\phi : \mathbb{R}^n \to \bar{\mathbb{R}}$ and a C^p $(p \geq 2)$ embedded submanifold $\mathcal{M}$ of $\mathbb{R}^n$. The function ϕ is said to be C^p-partly smooth at $x \in \mathcal{M}$ for $v \in \partial\phi(x)$ relative to $\mathcal{M}$ if

(i) Smoothness: ϕ restricted to $\mathcal{M}$ is C^p-smooth near x.

(ii) Prox-regularity: ϕ is prox-regular at x for v.

(iii) Sharpness: $\operatorname{par}\partial_p\phi(x) = N_{\mathcal{M}}(x)$, where ∂_p denotes the set of proximal subgradients of ϕ at point x, $\operatorname{par}\Omega$ is the subspace parallel to Ω, and $N_{\mathcal{M}}(x)$ is the normal space of $\mathcal{M}$ at x.

(iv) Continuity: There exists a neighborhood V of v such that the set-valued mapping $V \cap \partial\phi$ is inner semicontinuous at x relative to $\mathcal{M}$.

One usage of the partial smoothness is connecting the relative interior condition in (iii) with SC to derive certain smoothness in nonsmooth optimization [3]. The local error bound condition [49] is a powerful tool for analyzing local superlinear convergence in the absence of nonsingularity.

Definition 3. We say the local error bound condition holds for F if there exist $\gamma_l > 0$ and $\varepsilon_l > 0$ such that for all $\boldsymbol{w}$ with $\operatorname{dist}(\boldsymbol{w}, \boldsymbol{W}_*) \le \varepsilon_l$, it holds that

$$\|F(\boldsymbol{w})\| \ge \gamma_l \operatorname{dist}(\boldsymbol{w}, \boldsymbol{W}_*), \tag{2.18}$$

where $\boldsymbol{W}_*$ is the solution set of $F(\boldsymbol{w}) = 0$ and $\operatorname{dist}(\boldsymbol{w}, \boldsymbol{W}_*) := \arg\min_{\boldsymbol{u}\in\boldsymbol{W}_*} \|\boldsymbol{w} - \boldsymbol{u}\|$.

Using the partial smoothness and local error bound condition, we have the following local superlinear convergence result [13, Theorem 2].

Theorem 2. *Suppose Assumption 1 holds and $p(\boldsymbol{x}), f(\mathcal{B}(\boldsymbol{x}))$ are partial smooth. For any optimal solution $\boldsymbol{w}_*$, if the SC is satisfied at $\boldsymbol{w}_*$, F defined by* (2.8) *is locally C^{p-1}-smooth in a neighborhood of $\boldsymbol{w}_*$. Furthermore, if $\boldsymbol{w}^k$ is close enough to $\boldsymbol{w}_* \in \boldsymbol{W}^*$ where the SC and the local error bound condition* (2.18) *hold, then* (2.15) *always holds with $i = 0$ and $\boldsymbol{w}^k$ converges to $\boldsymbol{w}_*$ Q-superlinearly.*

Notably, the partial smoothness and Slater's condition are commonly encountered in various applications. Even though the local error bound condition may appear restrictive, such a condition is satisfied when the functions p and f are piecewise linear-quadratic, such as ℓ_1, ℓ_∞ norm and box constraint.

2.3 An efficient implementation to solve the linear system

Ignoring the subscript k, the linear system (2.13) can be represented by:

$$\begin{pmatrix} \mathcal{H}_{\mathbf{11}} + \tau\mathcal{I} & \mathcal{H}_{\mathbf{12}} \\ -\mathcal{H}_{\mathbf{12}}^T & \mathcal{H}_{\mathbf{22}} + \tau\mathcal{I} \end{pmatrix} \begin{pmatrix} \boldsymbol{d_1} \\ \boldsymbol{d_2} \end{pmatrix} = - \begin{pmatrix} \tilde{F}_{\mathbf{1}} \\ \tilde{F}_{\mathbf{2}} \end{pmatrix}, \tag{2.19}$$

where $\tilde{F} = F - \boldsymbol{\varepsilon}, F = (F_1, F_2), F_1 = (F_{\boldsymbol{y}}, F_{\boldsymbol{z}}, F_{\boldsymbol{r}}, F_{\boldsymbol{v}}), F_2 = (F_{\boldsymbol{x}_1}, F_{\boldsymbol{x}_2}, F_{\boldsymbol{x}_3}, F_{\boldsymbol{x}_4}), \boldsymbol{d}_1 = (\boldsymbol{d_y}, \boldsymbol{d_z}, \boldsymbol{d_r}, \boldsymbol{d_v}), \boldsymbol{d}_2 = (\boldsymbol{d}_{\boldsymbol{x}_1}, \boldsymbol{d}_{\boldsymbol{x}_2}, \boldsymbol{d}_{\boldsymbol{x}_3}, \boldsymbol{d}_{\boldsymbol{x}_4})$. For a given $\boldsymbol{d_1}$, the direction $\boldsymbol{d_2}$ can be calculated by

$$\boldsymbol{d_2} = (\mathcal{H}_{\mathbf{22}} + \tau\mathcal{I})^{-1}(\mathcal{H}_{\mathbf{12}}^\top \boldsymbol{d_1} - F_{\mathbf{2}}). \tag{2.20}$$

Hence, the linear equation (2.19) reduces to a linear system with respect to $\boldsymbol{d_1}$:

$$\widetilde{\mathcal{H}}_{\mathbf{11}} \boldsymbol{d_1} = -\widetilde{F}_{\mathbf{1}}, \tag{2.21}$$

where $\widetilde{F}_{\mathbf{1}} := \mathcal{H}_{\mathbf{12}}(\mathcal{H}_{\mathbf{22}} + \tau\mathcal{I})^{-1}\tilde{F}_{\mathbf{2}} - \tilde{F}_{\mathbf{1}}$ and $\widetilde{\mathcal{H}}_{\mathbf{11}} := (\mathcal{H}_{\mathbf{11}} + \mathcal{H}_{\mathbf{12}}(\mathcal{H}_{\mathbf{22}} + \tau\mathcal{I})^{-1}\mathcal{H}_{\mathbf{12}}^\top + \tau\mathcal{I})$. The definition of $\mathcal{H}_{\mathbf{12}}$ in (2.11) yields

$$\widetilde{\mathcal{H}}_{\mathbf{11}} = (\mathcal{A}, \mathcal{B}, \mathcal{I}, \mathcal{Q})^{\mathrm{T}} \, \overline{D}_p \, (\mathcal{A}, \mathcal{B}, \mathcal{I}, \mathcal{Q}) + \sigma \operatorname{blkdiag}(\overline{D}_{\Pi_1}, \overline{D}_{\mathrm{F}}, \overline{D}_{\Pi_2}, \mathcal{Q}), \tag{2.22}$$

where blkdiag denotes the block diagonal operator, $\overline{D}_p = \sigma D_p + \widetilde{D}_p, \widetilde{D}_p = D_p(\frac{1}{\sigma}(\mathcal{I} - D_p) + \tau\mathcal{I})^{-1} D_p, \overline{D}_{\Pi_1}, \overline{D}_{\mathrm{F}}$ and $\overline{D}_{\Pi_2}$ are defined analogously. If the problem has more than one primal variable, we can solve the linear system (2.21) using iterative methods. According to (2.22), $(\mathcal{A}, \mathcal{B}, \mathcal{I}, \mathcal{Q})\, \boldsymbol{d}_1$ can be computed first

and shared among all components. If the corresponding solution is sparse or low-rank, then the special structures of $\overline{D}_p$ can further be used to improve the computational efficiency. Furthermore, if $\widetilde{\mathcal{H}}_{11}$ only has one variable, we can solve the equation (2.21) using direct methods, such as Cholesky factorization method.

We also note that some variables in $\boldsymbol{w}$ may not exist if the function or constraint does not exist in (1.1). The existence condition of variables is listed in the following.

- $\boldsymbol{y}$ exists if and only if $\mathcal{P}_2$ is nontrivial. $\boldsymbol{x}_1$ exist if and only if $\mathcal{P}_2$ is not a singleton set.
- $\boldsymbol{z}$ exists if and only if f exists. $\boldsymbol{x}_2$ exists if and only if f exists and is nonsmooth.
- $\boldsymbol{r}$ and $\boldsymbol{x}_3$ exist if and only if $\mathcal{P}_1$ is nontrivial.
- $\boldsymbol{v}$ exists if and only if $\mathcal{Q}$ is nontrivial.

For example, for the Lasso problem, $p(\boldsymbol{x}) = \|\boldsymbol{x}\|_1, f(\mathcal{B}(\boldsymbol{x})) = \frac{1}{2}\|\mathcal{B}(\boldsymbol{x}) - \boldsymbol{b}\|^2, \boldsymbol{c} = \boldsymbol{0}, \mathcal{Q} = \boldsymbol{0}, \mathcal{P}_1 = \mathcal{P}_2 = \oslash$. The valid variables are $\boldsymbol{z}$ and $\boldsymbol{x}_4$, i.e., one primal variable and one dual variable. Consequently, for problems where $\mathcal{H}_{11}$ only has one primal variable, such as Lasso, and SOCP, we can solve the linear system using direct methods such as Cholesky factorization at low cost.

2.4 Practical implementations

To ensure that Algorithm 1 has a better performance on various problems, we present some implementation details of Algorithm 1 used to solve (2.9) in this section.

2.4.1 Line search for $\boldsymbol{d}^k$

In some cases, condition (2.15) may not be satisfied with the full regularized Newton step in (2.14). The sufficient decrease property (2.15) may be easier to satisfy when a line search strategy is used for problems such as Lasso-type problems. Specifically, we choose appropriate α and $\tilde{\boldsymbol{w}}^{k,i} = \boldsymbol{w}^k + \alpha \boldsymbol{d}^{k,i}$ such that condition

$$\|F(\tilde{\boldsymbol{w}}^{k,i})\| < \nu \max_{\max(1,k-\zeta+1)} \|F(\boldsymbol{w}^j)\| + \varsigma_k \tag{2.23}$$

holds, we then set $\boldsymbol{w}^{k+1} = \tilde{\boldsymbol{w}}^{k,i}$. If (2.23) is not satisfied after several line searches, then we set $\boldsymbol{w}^{k+1} = \bar{\boldsymbol{w}}^{k,i}$ with (2.16) being held. Since it needs one additional proximal operator calculation every time, the line search property is only effective for $p(x)$ whose proximal operator can be calculated efficiently.

2.4.2 Update regularization parameter κ

κ serves as the constant in the definition of $\tau_{k,i}$ when $i < i_{\max}$, which is of vital importance to control the quality of $\boldsymbol{w}^k$. When κ is small, the Newton equation is accurate, but $\boldsymbol{d}^k$ may not be a good direction. For an iterate $\boldsymbol{w}^k$, $\boldsymbol{d}_1^k$ and $\boldsymbol{d}_2^k$ are descent or ascent directions if for the corresponding primal and dual variables if $\langle \boldsymbol{d}_1^k, F_1 \rangle < 0$ and $\langle \boldsymbol{d}_2^k, F_2 \rangle < 0$, respectively. Taking into account this situation, we define the ratio

$$\rho_k := \frac{-\langle \boldsymbol{d}^k, F(\boldsymbol{w}^{k+1}) \rangle}{\|\boldsymbol{d}^k\|_2^2} \tag{2.24}$$

to decide whether $\boldsymbol{d}_k$ is a bad direction and how to update κ_k. If ρ_k is small, it is usually a signal of a bad Newton step and we increase κ_k. Otherwise, we decrease it. Specifically, the parameter κ_k is updated as

$$\kappa_{k+1} = \begin{cases} \max\{\gamma_1 \kappa_k, \underline{\tau}\}, & \text{if } \rho_k \geq \eta_2, \\ \gamma_2 \kappa_k, & \text{if } \eta_2 > \rho_k \geq \eta_1, \\ \min\{\gamma_3 \kappa_k, \bar{\tau}\}, & \text{otherwise}, \end{cases} \tag{2.25}$$

where $0 < \eta_1 \leq \eta_2, 0 < \gamma_1 < \gamma_2 < 1, \gamma_3 > 1$ are chosen parameters and $\underline{\tau}, \bar{\tau}$ are two predefined positive constants.

2.4.3 Update penalty parameter σ

We also adaptively adjust the penalty factor σ based on the primal and dual infeasibility. Specifically, if the primal infeasibility exceeds the dual infeasibility over a certain number of steps, we decrease σ; otherwise, we increase it. Specifically, we next show our strategies for how to update σ incorporating the iteration information. We mainly examine the ratios of primal and dual infeasibilities of the last few steps defined by

$$\omega^k = \frac{\text{geomean}_{k-l \leq j \leq k} \eta_P^j}{\text{geomean}_{k-l \leq j \leq k} \eta_D^j}, \tag{2.26}$$

where the primal infeasibility η_P and the dual infeasibility η_D are defined by

$$\eta_P = \frac{\|\mathcal{A}(\boldsymbol{x}) - \Pi_{\mathcal{P}_2}(\mathcal{A}(\boldsymbol{x}) - \boldsymbol{y})\|}{1 + \|\boldsymbol{x}\|}, \quad \text{and} \quad \eta_D = \frac{\|\mathcal{A}^*(\boldsymbol{y}) + \mathcal{B}^*(\boldsymbol{z}) + \boldsymbol{s} - \mathcal{Q}(\boldsymbol{v}) - \boldsymbol{c}\|}{1 + \|\boldsymbol{c}\|}, \tag{2.27}$$

and l is a hyperparameter. For every l steps, we check ω^k. If ω^k is larger (or smaller) than a constant δ, we decrease (or increase) the penalty parameter σ by a multiplicative factor γ (or $1/\gamma$) with $0 < \gamma < 1$. To prevent σ from becoming excessively large or small, upper and lower bounds are imposed on σ. This strategy has been demonstrated to be effective in solving SDP problems [27].

3 Properties of proximal operators

In this section, we demonstrate how to handle the shift term and the computational details of other proximal operators. According to (2.20), we need the explicit calculation process of $(\frac{1}{\sigma}(\mathcal{I} - D_p) + \tau\mathcal{I})^{-1}$ and $\sigma D_p + D_p(\frac{1}{\sigma}(\mathcal{I} - D_p) + \tau\mathcal{I})^{-1} D_p$. Furthermore, if $\boldsymbol{x}$ and $\mathcal{B}(\boldsymbol{x})$ are replaced by $\boldsymbol{x} - \boldsymbol{b}_1$ or $\mathcal{B}(\boldsymbol{x}) - \boldsymbol{b}_2$, respectively, the variables need to be corrected by a shift term. Some proximal operators, such as the semidefinite cone or the ℓ_∞ norm, are already known in the literature.

3.1 Handling `shift` term

For problems that have a shift term such as $p(\boldsymbol{x} - \boldsymbol{b}_1)$ or $f(\mathcal{B}(\boldsymbol{x}) - \boldsymbol{b}_2)$, the corresponding dual problem of (1.1) is

$$\begin{aligned} \min_{\boldsymbol{y},\boldsymbol{z},\boldsymbol{s},\boldsymbol{r},\boldsymbol{v}} \quad & \delta^*_{\mathcal{P}_2}(-\boldsymbol{o}) + f^*(-\boldsymbol{q}) - \langle \boldsymbol{b}_2, \boldsymbol{q} \rangle - \langle \boldsymbol{b}_1, \boldsymbol{s} \rangle + p^*(-\boldsymbol{s}) + \frac{1}{2} \langle \mathcal{Q}\boldsymbol{v}, \boldsymbol{v} \rangle + \delta^*_{\mathcal{P}_1}(-\boldsymbol{t}), \\ \text{s.t.} \quad & \mathcal{A}^*(\boldsymbol{y}) + \mathcal{B}^*\boldsymbol{z} + \boldsymbol{s} - \mathcal{Q}\boldsymbol{v} + \boldsymbol{r} = \boldsymbol{c}, \quad \boldsymbol{y} = \boldsymbol{o}, \quad \boldsymbol{z} = \boldsymbol{q}, \quad \boldsymbol{r} = \boldsymbol{t}. \end{aligned} \tag{3.1}$$

If f^* is differentiable, the gradient with respect to $\boldsymbol{q}$ is $-\nabla f^*(-\boldsymbol{q}) - \boldsymbol{b}_2$. If f is nonsmooth, it follows from the property of the proximal operator that $\boldsymbol{q} = \text{prox}_{f^*/\sigma}(\boldsymbol{x}_2/\sigma - \boldsymbol{z} - \boldsymbol{b}_2/\sigma)$. Hence, we only need to replace the $\text{prox}_{\sigma f}(\boldsymbol{x}_2 - \sigma\boldsymbol{z})$ in (2.7) with $\text{prox}_{\sigma f}(\boldsymbol{x}_2 - \sigma(\boldsymbol{z} - \boldsymbol{b}_2)) - \boldsymbol{b}_2$. Similarly, for $p^*(-\boldsymbol{s})$, the corresponding term $\text{prox}_{\sigma p}(\boldsymbol{x}_4 + \sigma(\mathcal{A}^*(\boldsymbol{y}) + \mathcal{B}^*\boldsymbol{z} - \mathcal{Q}\boldsymbol{v} + \boldsymbol{r} - \boldsymbol{c}))$ is replaced by $\text{prox}_{\sigma p}(\boldsymbol{x}_4 + \sigma(\mathcal{A}^*(\boldsymbol{y}) + \mathcal{B}^*\boldsymbol{z} - \mathcal{Q}\boldsymbol{v} + \boldsymbol{r} - \boldsymbol{c} - \boldsymbol{b}_1)) + \boldsymbol{b}_1$. Hence, we do not need to introduce a slack variable when adding a shift term $\boldsymbol{b}_1$ or $\boldsymbol{b}_2$ to $p(\boldsymbol{x})$ or $f(\mathcal{B}(\boldsymbol{x}))$.

3.2 ℓ_2 norm regularizer

For the ℓ_2 norm, i.e., $p(\boldsymbol{x}) = \lambda\|\boldsymbol{x}\|_2$, its proximal operator is $\mathrm{prox}_{\lambda\|\cdot\|_2}(\boldsymbol{x}) = \begin{cases} \boldsymbol{x} - \lambda\boldsymbol{x}/\|\boldsymbol{x}\|, & \text{if } \|\boldsymbol{x}\| > \lambda, \\ 0, & \text{otherwise.} \end{cases}$.
Consequently, one generalized Jacboian D of the ℓ_2 norm is

$$D = \begin{cases} I - \lambda(I - \frac{\boldsymbol{x}\boldsymbol{x}^{\mathrm{T}}}{\|\boldsymbol{x}\|^2})/\|\boldsymbol{x}\|, & \text{if } \|\boldsymbol{x}\| > \lambda, \\ 0, & \text{otherwise,} \end{cases}$$

It follows from the SMW formula $(A - uu^{\mathrm{T}})^{-1} = A^{-1} + \frac{A^{-1}uu^{\mathrm{T}}A^{-1}}{1-u^{\mathrm{T}}A^{-1}u}$ that for $D \in \partial\mathrm{prox}_{\lambda\|\cdot\|_2}(\boldsymbol{x})$,

$$\left(\tau I + \frac{1}{\sigma}(I - D)\right)^{-1} = \left(\tau I + \lambda(I - \boldsymbol{x}\boldsymbol{x}^{\mathrm{T}}/\|\boldsymbol{x}\|^2)/(\sigma\|\boldsymbol{x}\|)\right)^{-1} = \frac{1}{\tau + \frac{\lambda}{\sigma\|\boldsymbol{x}\|}}\left(I + \frac{\lambda}{\sigma\tau\|\boldsymbol{x}\|}\boldsymbol{x}^{\mathrm{T}}\boldsymbol{x}/\|\boldsymbol{x}\|^2\right).$$

Hence, the following qualities hold:

$$\begin{aligned} D\left(\tau I + \frac{1}{\sigma}(I - D)\right)^{-1} &= \left(\left(1 - \frac{\lambda}{\|\boldsymbol{x}\|}\right)I + \left(\frac{1}{\tau\sigma} + 1\right)\frac{\lambda}{\|\boldsymbol{x}\|}\boldsymbol{x}\boldsymbol{x}^{\mathrm{T}}/\|\boldsymbol{x}\|^2\right)\left(\frac{1}{\tau + \frac{\lambda}{\sigma\|\boldsymbol{x}\|}}\right), \\ \overline{D} = \sigma D + D\left(\tau I + \frac{1}{\sigma}(I - D)\right)^{-1} D &= \left(\sigma\left(1 - \frac{\lambda}{\|\boldsymbol{x}\|}\right) + \left(\frac{1}{\tau + \frac{\lambda}{\sigma\|\boldsymbol{x}\|}}\right)\left(1 - \frac{\lambda}{\|\boldsymbol{x}\|}\right)\right)I \\ &\quad + \left(\left(\frac{1}{\tau + \frac{\lambda}{\sigma\|\boldsymbol{x}\|}}\right)\left(2\frac{\lambda}{\|\boldsymbol{x}\|} + \frac{\lambda}{\|\boldsymbol{x}\|\sigma\tau} - \frac{\lambda^2}{\|\boldsymbol{x}\|^2}\right) + \frac{\sigma\lambda}{\|\boldsymbol{x}\|}\right)\boldsymbol{x}\boldsymbol{x}^{\mathrm{T}}/\|\boldsymbol{x}\|^2. \end{aligned} \tag{3.2}$$

Consequently, the operators in (3.2) can be represented as an identity matrix multiplied by a constant plus a rank-one correction. For $p(\boldsymbol{x}) = \delta_{\|\boldsymbol{x}\|_2 \le \lambda}(\boldsymbol{x})$, the derivation is similar and omitted.

3.3 Second-order cone

Let $Q^n \subseteq \mathbb{R}^n$ denote the n-dimensional second-order cone (SOC), defined as

$$Q^n = \{\boldsymbol{x} \in \mathbb{R}^n \,:\, \|\bar{\boldsymbol{x}}\| \le x_0\}.$$

Here, a vector $\boldsymbol{x} \in \mathbb{R}^n$ is partitioned as $\boldsymbol{x} = [x_0; \bar{\boldsymbol{x}}]$, where $x_0 \in \mathbb{R}$ is its scalar part and $\bar{\boldsymbol{x}} \in \mathbb{R}^{n-1}$ is its vector part. For any $\boldsymbol{x}$ in the interior of the cone, $\boldsymbol{x} \in \mathrm{int}(Q^n)$, its determinant is given by $\det(\boldsymbol{x}) = x_0^2 - \|\bar{\boldsymbol{x}}\|^2$. If the determinant is non-zero, its inverse is $\boldsymbol{x}^{-1} = \frac{1}{\det(\boldsymbol{x})}[x_0; -\bar{\boldsymbol{x}}]$. A generalized Jacobian D associated with the second-order cone is given by:

$$D = \begin{cases} I, & \text{if } x_0 \ge \|\bar{x}\|, \\ 0, & \text{if } x_0 \le -\|\bar{x}\|, \\ \frac{1}{2}\begin{bmatrix} 1 & \frac{\bar{x}^\top}{\|\bar{x}\|} \\ \frac{\bar{x}}{\|\bar{x}\|} & \left(1 + \frac{x_0}{\|\bar{x}\|}\right)I - \frac{x_0}{\|\bar{x}\|^3}\bar{x}\bar{x}^\top \end{bmatrix}, & \text{otherwise.} \end{cases} \tag{3.3}$$

For the third case, the generalized Jacobian of the second-order cone admits a low-rank decomposition:

$$D = \frac{1}{2}\left(1 + \frac{x_0}{\|\bar{x}\|}\right)I + \frac{1}{2}\left(1 - \frac{x_0}{\|\bar{x}\|}\right)\begin{bmatrix} \frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2}\frac{\bar{x}}{\|\bar{x}\|} \end{bmatrix}\begin{bmatrix} \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2}\frac{\bar{x}^T}{\|\bar{x}\|} \end{bmatrix} + \left(-\frac{1}{2}\right)\left(1 + \frac{x_0}{\|\bar{x}\|}\right)\begin{bmatrix} \frac{\sqrt{2}}{2} \\ -\frac{\sqrt{2}}{2}\frac{\bar{x}}{\|\bar{x}\|} \end{bmatrix}\begin{bmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2}\frac{\bar{x}^T}{\|\bar{x}\|} \end{bmatrix}.$$

Define the logarithmic barrier function for any $x \in Q^n$ by $g : \mathbb{R}^n \mapsto \mathbb{R}$ with

$$g(x) = \begin{cases} -\frac{1}{2}\log(\det(x)), & \text{if } \|x_0\| > \bar{x}, \\ +\infty, & \text{otherwise.} \end{cases}$$

We note that $\lim_{\mu\to 0} \mu g(x) = \delta_{Q^n}(x)$. For SOCP, dealing with the smooth barrier functions may yield better numerical results than dealing with the conic constraints directly. In this case, we use a barrier function $p(\boldsymbol{x}) = \mu g(\boldsymbol{x})$ to replace the cone constraint function $p(\boldsymbol{x}) = \delta_Q(\boldsymbol{x})$, where $\mu > 0$. For the smoothing function $\mu g(x)$, the following lemma holds.

Lemma 2. *(i) The proximal mapping of $\mu g(x)$ is given by $prox_{\mu g} : \mathbb{R}^n \mapsto int(\mathcal{Q}^n)$ with*

$$\text{prox}_{\mu g}(z) = \begin{bmatrix} \frac{1}{2}\left(z_0 + \sqrt{\frac{1}{2}(\|z\|^2 + 4\mu + \Delta)}\right) \\ \frac{\bar{z}}{2}\left(1 + \frac{\sqrt{2}z_0}{\sqrt{\|z\|^2+4\mu+\Delta}}\right) \end{bmatrix}, \quad z \in \mathbb{R}^n, \tag{3.4}$$

where $\Delta = \sqrt{\det(z)^2 + 8\mu\|z\|^2 + 16\mu^2}$. Furthermore, the inverse function of the proximal mapping is given by $prox_{\mu g}^{-1} : int(\mathcal{Q}^n) \to \mathbb{R}$ with

$$\text{prox}_{\mu g}^{-1}(x) = x - \mu x^{-1}, \quad x \in int(\mathcal{Q}^n). \tag{3.5}$$

(ii) The projection function is the limit of the proximal mapping as μ approaches 0, i.e.,

$$\lim_{\mu\to 0} prox_{\mu g}(z) = \Pi_K(z), \quad z \in \mathbb{R}^n.$$

(iii) For $z \in \mathbb{R}^n$, let $x = prox_{\mu g}(z)$. The inverse matrix of the derivative of the proximal mapping at the point z is given by

$$(\partial_z \text{prox}_{\mu g}(z))^{-1} = I - \mu\partial_x(x^{-1}),$$

where $\partial_x(x^{-1}) = \frac{1}{\det(x)}\begin{bmatrix} 1 & \\ & -I_{n-1} \end{bmatrix} - 2(x^{-1})(x^{-1})^\top$.

(iv) The derivative of the proximal mapping at the point z is given by

$$\partial_z \text{prox}_{\mu g}(z) = \begin{bmatrix} \frac{\det(x)}{\det(x)-\mu} & \\ & \frac{\det(x)}{\det(x)+\mu} I_{n-1} \end{bmatrix} - \frac{2\mu\det(x)\begin{bmatrix} \frac{x_0}{\det(x)-\mu} \\ \frac{-\bar{x}}{\det(x)+\mu} \end{bmatrix}\begin{bmatrix} \frac{x_0}{\det(x)+\mu} \\ \frac{-\bar{x}}{\det(x)+\mu} \end{bmatrix}^{\mathrm{T}}}{\det(x) + 2\mu\left(\frac{x_0^2}{\det(x)-\mu} + \frac{\|\tilde{x}\|^2}{\det(x)+\mu}\right)} := \Lambda + auu^{\mathrm{T}},$$

where $\Lambda = \begin{bmatrix} a_0 & \\ & a_1 I_{n-1} \end{bmatrix}$ *and $a_0, a_1 \in \mathbb{R}$ are constants.*

Proof. (i) Given $z \in \mathbb{R}^n$, it follows from the definition of the proximal mapping that $x = \text{prox}_{\mu g}(z)$ is the optimal point of the following minimization problem

$$\min_{x\in\text{int}(\mathcal{Q}^n)} f(x) = \frac{1}{2}\|z - x\|^2 - \frac{1}{2}\mu\log\det(x).$$

$x \in \text{int}(\mathcal{Q}^n)$ implies $\det(x) > 0$. The optimality condition $x - z - \mu x^{-1} = 0$ is equivalent to

$$\text{prox}_{\mu g}^{-1}(x) = z = x - \mu x^{-1} \Longleftrightarrow \begin{cases} z_0 = x_0 - \frac{\mu}{\det(x)}x_0, \\ \bar{z} = \bar{x} + \frac{\mu}{\det(x)}\bar{x}. \end{cases} \tag{3.6}$$

Hence (3.5) holds. To derive the expression of $\operatorname{prox}_{\mu g}(z)$, we consider the following two cases. If $\frac{\mu}{\det(x)} = 1$, then $z_0 = 0, \bar{x} = \frac{1}{2}\bar{z}$ and $x_0 = \sqrt{\det(x) + \|\bar{x}\|^2} = \sqrt{\frac{1}{4}\|\bar{z}\|^2 + \mu}$. Otherwise $\frac{\mu}{\det(x)} \neq 1$, then $z_0 \neq 0$. Provided with this condition, $\nabla f(x) = 0$ is equivalent to

$$x_0 = \frac{z_0}{1-\rho}, \quad \bar{x} = \frac{\bar{z}}{1+\rho}, \tag{3.7}$$

where $\rho = \frac{\mu}{\det(x)} > 0$. Combined with the identity that $\det(x) = x_0^2 - \|\bar{x}\|^2$, we see ρ is the root of the polynomial equation

$$\frac{z_0^2}{(1-\rho)^2} - \frac{\|\bar{z}\|^2}{(1+\rho)^2} = \frac{\mu}{\rho} \Longleftrightarrow \frac{z_0^2}{\rho + \frac{1}{\rho} - 2} - \frac{\|\bar{z}\|^2}{\rho + \frac{1}{\rho} + 2} = \mu.$$

Let $r = \rho + \frac{1}{\rho}$. Note that $r > 2$. By solving the above equation, we have

$$r = \frac{\det(z) + \Delta}{2\mu}, \quad \rho = \begin{cases} \frac{r - \sqrt{r^2-4}}{2} & \text{if } z_0 > 0, \\ \frac{r + \sqrt{r^2-4}}{2} & \text{if } z_0 < 0, \end{cases}$$

where $\Delta = \sqrt{\det(z)^2 + 8\mu\|z\|^2 + 16\mu^2}$. Subsequently, we take ρ into (3.6) and have for $z_0 > 0$, $\rho < 1$,

$$\begin{aligned} x_0 &= \frac{z_0}{1-\rho} = \frac{z_0}{2}\left(1 + \sqrt{\frac{r+2}{r-2}}\right) = \frac{1}{2}\left(z_0 + \sqrt{\frac{1}{2}(\|z\|^2 + 4\mu + \Delta)}\right), \\ \bar{x} &= \frac{\bar{z}}{1+\rho} = \frac{\bar{z}}{2}\left(1 + \sqrt{\frac{r-2}{r+2}}\right) = \frac{\bar{z}}{2}\left(1 + \frac{\sqrt{2}z_0}{\sqrt{\|z\|^2 + 4\mu + \Delta}}\right). \end{aligned}$$

For $z_0 < 0$, $\rho > 1$,

$$\begin{aligned} x_0 &= \frac{z_0}{1-\rho} = \frac{z_0}{2}\left(1 - \sqrt{\frac{r+2}{r-2}}\right) = \frac{1}{2}\left(z_0 + \sqrt{\frac{1}{2}(\|z\|^2 + 4\mu + \Delta)}\right), \\ \bar{x} &= \frac{\bar{z}}{1+\rho} = \frac{\bar{z}}{2}\left(1 - \sqrt{\frac{r-2}{r+2}}\right) = \frac{\bar{z}}{2}\left(1 + \frac{\sqrt{2}z_0}{\sqrt{\|z\|^2 + 4\mu + \Delta}}\right). \end{aligned}$$

Therefore (3.4) holds for any $z \in \mathbb{R}^n$.

(ii) Let $\mu \to 0$, then $\Delta \to |\det(z)|$. Hence, it follows that

$$\lim_{\mu\to 0} \operatorname{prox}_{\mu g}(z) = \begin{bmatrix} \frac{1}{2}\left(z_0 + \sqrt{\frac{1}{2}(\|z\|^2 + |\det(z)|)}\right) \\ \frac{\bar{z}}{2}\left(1 + \frac{\sqrt{2}z_0}{\sqrt{\|z\|^2 + |\det(z)|}}\right) \end{bmatrix} = \begin{cases} z, & \text{if } z_0 \geq \|\bar{z}\|, \\ 0, & \text{if } z_0 \leq -\|\bar{z}\|, \\ \begin{bmatrix} \frac{1}{2}(z_0 + \|\bar{z}\|) \\ \frac{\bar{z}}{2}\left(1 + \frac{z_0}{\|\bar{z}\|}\right) \end{bmatrix}, & \text{if } -\|\bar{z}\| < z_0 < \|\bar{z}\| \end{cases} = \Pi_K(z), \quad z \in \mathbb{R}^n.$$

(iii) Note that $\operatorname{prox}_{\mu g}^{-1}$ is a single-valued mapping. By the inverse function theorem, it holds that

$$(\partial_z \operatorname{prox}_{\mu g}(z))^{-1} = \partial_x \operatorname{prox}_{\mu g}^{-1}(x) = \partial_x\left(x - \mu x^{-1}\right) \overset{(3.6)}{=} I - \mu\partial_x(x^{-1}) = I - \mu\left(\frac{1}{\det(x)}\begin{bmatrix} 1 & \\ & -I_{n-1} \end{bmatrix} - 2(x^{-1})(x^{-1})^\top\right),$$

where the last equation is obtained from the following derivation:

$$\partial_x\left(x^{-1}\right) = \partial_x\left(\frac{1}{\det(x)}\begin{bmatrix} x_0 \\ -\bar{x} \end{bmatrix}\right) = -\frac{1}{\det(x)^2}\partial_x\left(\det(x)\right)\begin{bmatrix} x_0 \\ -\bar{x} \end{bmatrix}^\top + \frac{1}{\det(x)}\begin{bmatrix} 1 & \\ & -I_{n-1} \end{bmatrix}$$

$$= -\frac{2}{\det(x)^2}\begin{bmatrix} x_0 \\ -\bar{x}\end{bmatrix}\begin{bmatrix} x_0 \\ -\bar{x}\end{bmatrix}^\top + \frac{1}{\det(x)}\begin{bmatrix} 1 & \\ & -I_{n-1}\end{bmatrix} = -2\left(x^{-1}\right)\left(x^{-1}\right)^\top + \frac{1}{\det(x)}\begin{bmatrix} 1 & \\ & -I_{n-1}\end{bmatrix}.$$

(iv) Let $\rho = \frac{\mu}{\det(x)}$, $\Lambda = I - \rho\begin{bmatrix} 1 & \\ & -I_{n-1}\end{bmatrix}$, $v = x^{-1}$. Then $(\partial_z \operatorname{prox}_{\mu g}(z))^{-1} = \Lambda + 2\mu v v^\top$. By the SMW formula, we have

$$\partial_z \operatorname{prox}_{\mu g}(z) = \Lambda^{-1} - \frac{2\mu\Lambda^{-1}vv^\top\Lambda^{-1}}{1+2\mu v^\top\Lambda^{-1}v} = \begin{bmatrix} \frac{1}{1-\rho} & \\ & \frac{1}{1+\rho}I_{n-1}\end{bmatrix} - \frac{2\mu\begin{bmatrix} \frac{x_0}{1-\rho} \\ \frac{-\bar{x}}{1+\rho}\end{bmatrix}\begin{bmatrix} \frac{x_0}{1-\rho} \\ \frac{-\bar{x}}{1+\rho}\end{bmatrix}^\top}{\det(x)^2 + 2\mu\left(\frac{x_0^2}{1-\rho} + \frac{\|\bar{x}\|^2}{1+\rho}\right)}$$

$$= \begin{bmatrix} \frac{1}{1-\rho} & \\ & \frac{1}{1+\rho}I_{n-1}\end{bmatrix} - \frac{2\begin{bmatrix} \frac{x_0}{1-\rho} \\ \frac{-\bar{x}}{1+\rho}\end{bmatrix}\begin{bmatrix} \frac{x_0}{1-\rho} \\ \frac{-\bar{x}}{1+\rho}\end{bmatrix}^\top}{\frac{\det(x)}{\rho} + 2\left(\frac{x_0^2}{1-\rho} + \frac{\|\bar{x}\|^2}{1+\rho}\right)},$$

It follows from $\rho = 1$ or $\rho \to 1$ that

$$\partial_z \operatorname{prox}_{\mu g}(z) = \frac{1}{2}\begin{bmatrix} 1 & \frac{1}{x_0}\bar{x}^\top \\ \frac{1}{x_0}\bar{x} & I_{n-1}\end{bmatrix}.$$

This completes the proof. □

It follows from Lemma 2 that $\frac{1}{\sigma}(I-D) + \tau I = \frac{1}{\sigma}\left(\begin{bmatrix} \tilde{a}_0 & \\ & \tilde{a}_1 I\end{bmatrix} - auu^{\mathrm{T}}\right) := \frac{1}{\sigma}\left(\Lambda_1 - auu^{\mathrm{T}}\right), \Lambda_1 = (1+\sigma\tau)I - \Lambda$. Consequently, we can obtain from the SMW formula that $(\frac{1}{\sigma}(I-D)+\tau I)^{-1} = \sigma(\Lambda_1^{-1} + c\Lambda_1^{-1}uu^{\mathrm{T}}\Lambda_1^{-1})$, where $c = \frac{a}{1-au^{\mathrm{T}}\Lambda_1^{-1}u}$ is a constant. Hence, the following equality holds.

$$\begin{aligned}
&\sigma D + D\left(\frac{1}{\sigma}(I-D)+\tau I\right)^{-1} D \\
=&\sigma\left(\Lambda + auu^{\mathrm{T}}\right) + \sigma(\Lambda + auu^{\mathrm{T}})(\Lambda_1^{-1} + c\Lambda_1^{-1}uu^{\mathrm{T}}\Lambda_1^{-1})(\Lambda + auu^{\mathrm{T}}) \\
=&\sigma\left(\Lambda\Lambda_1^{-1}\Lambda + \Lambda + c\Lambda\Lambda_1^{-1}uu^{\mathrm{T}}\Lambda_1^{-1}\Lambda + a(1+c\gamma)\Lambda\Lambda_1^{-1}uu^{\mathrm{T}} + a(1+c\gamma)uu^{\mathrm{T}}\Lambda_1^{-1}\Lambda + \left(a + a^2\gamma + a^2c\gamma^2\right)uu^{\mathrm{T}}\right) \\
=&\sigma\left(\tilde{\Lambda} + \begin{bmatrix} b_0u_0^2 & b_1u_0u_1^{\mathrm{T}} \\ b_1u_0u_1 & b_2u_1u_1^{\mathrm{T}}\end{bmatrix}\right),
\end{aligned}$$

where $\tilde{\Lambda} = \Lambda\Lambda_1^{-1}\Lambda + \Lambda, \gamma = u^{\mathrm{T}}\Lambda_1^{-1}u$, and b_0, b_1, b_2 are constants. Denote $\Lambda_1^{-1}\Lambda = \begin{bmatrix} c_0 & \\ & c_1 I\end{bmatrix}$, we have $b_0 = cc_0^2 + 2a(1+c\gamma)c_0 + a + a^2\gamma + a^2c\gamma^2, b_1 = cc_0c_1 + a(1+c\gamma)(c_0+c_1) + a + a^2\gamma + a^2c\gamma^2, b_2 = cc_1^2 + 2a(1+c\gamma)c_1 + a + a^2\gamma + a^2c\gamma^2$. Consequently, let $\tilde{u} = [b_1/b_2u_0; u_1]$, $b_2\tilde{u}\tilde{u}^{\mathrm{T}} = \begin{bmatrix} b_1^2/b_2u_0^2 & b_1u_0u_1^{\mathrm{T}} \\ b_1u_0u_1 & b_2u_1u_1^{\mathrm{T}}\end{bmatrix}$, it follows that

$$\overline{D} = \sigma D + D\left(\frac{1}{\sigma}(I-D)+\tau I\right)^{-1} D = \sigma\left(\tilde{\Lambda} + \begin{bmatrix} (b_0 - b_1^2/b_2)u_0^2 & 0 \\ 0 & 0\end{bmatrix}\right) + \sigma b_2\tilde{u}\tilde{u}^{\mathrm{T}}.$$

Hence, the linear system can be represented as a diagonal matrix plus a rank-one matrix, which is significant in constructing the Schur matrix when solving the linear system using direct methods such as Cholesky factorization.

3.4 Spectral functions

The spectral type functions include $p(\boldsymbol{X}) = \lambda\|\boldsymbol{X}\|_*, \lambda\|\boldsymbol{X}\|_2$ and $\delta_{\mathbb{S}^n_+}(\boldsymbol{X})$. For more details of the generalized Jacboian of spectral functions, we refer the readers to [41]. We present a non-exhaustive introduction to the usually used spectral function in the following. For a given $\boldsymbol{X} \in \mathbb{R}^{n_1\times n_2}$, let the singular value decomposition of $\boldsymbol{X}$ denoted by $\boldsymbol{X} = U\Sigma V^{\mathrm{T}}$, then the proximal operator of $\lambda\|\boldsymbol{X}\|_*$ can be presented by:

$$\mathrm{prox}_{\lambda\|\cdot\|_*}(\boldsymbol{X}) = U\mathrm{diag}\left(T_\lambda(\sigma(\boldsymbol{X}))\right)V^T, \tag{3.8}$$

where $T_\lambda(\cdot)$ denotes the soft shrinkage operator. Without loss of generality, we consider the case that $n_2 \geq n_1$. Let $V = [V_1, V_2]$ with $V_1 \in \mathbb{R}^{n_1\times n_1}$ and $V_2 \in \mathbb{R}^{n_1\times(n_2-n_1)}$, then one generalized Jacobian D of (3.8) is

$$D(G) = U\left[\frac{\Omega^\mu_{\sigma,\sigma} + \Omega^\mu_{\sigma,-\sigma}}{2} \odot G_1 + \frac{\Omega^\mu_{\sigma,\sigma} - \Omega^\mu_{\sigma,-\sigma}}{2} \odot G_1^\top, (\Omega^\mu_{\sigma,0} \odot (G_2))\right]V^\top, \tag{3.9}$$

where $\odot$ denotes the Hadamard product, $\sigma = [\sigma^{(1)}; \cdots; \sigma^{(m)}] \in \mathbb{R}^m$ is the singular value of $\boldsymbol{X}$, $G_1 = U^\top G V_1 \in \mathbb{R}^{n_1\times n_1}, G_2 = U^\top G V_2 \in \mathbb{R}^{n_1\times(n_2-n_1)}$ and $\Omega^\lambda_{\sigma,\sigma}$ is defined by:

$$(\Omega^\lambda_{\sigma,\sigma})_{ij} := \begin{cases} \partial_B \mathrm{prox}_{\lambda\|\cdot\|_1}(\sigma_i), & \text{if } \sigma_i = \sigma_j, \\ \left\{\frac{\mathrm{prox}_{\lambda\|\cdot\|_1}(\sigma_i) - \mathrm{prox}_{\lambda\|\cdot\|_1}(\sigma_j)}{\sigma_i - \sigma_j}\right\}, & \text{otherwise.} \end{cases} \tag{3.10}$$

For $p(\boldsymbol{X}) = \lambda\|\boldsymbol{X}\|_2$, its proximal operator can be represented by

$$\mathrm{prox}_{\lambda\|\cdot\|_2}(\boldsymbol{X}) = U\mathrm{diag}\left(\lambda(\boldsymbol{X}) - \lambda P_{\Delta_n}(\lambda(\boldsymbol{X})/\lambda)\right)V^T, \tag{3.11}$$

where P_{Δ_n} denotes the projection onto simplex unit $\Delta_n := \{\boldsymbol{x} \in \mathbb{R}^n | \mathbf{1}^{\mathrm{T}}\boldsymbol{x} = 1, \boldsymbol{x} \geq 0\}$. Hence, the generalized Jacobian of (3.11) is (3.9) with

$$(\Omega^\lambda_{\sigma,\sigma})_{ij} := \begin{cases} \partial_B(\mathrm{prox}_{\lambda\|\cdot\|_\infty}(\sigma))_i, & \text{if } \sigma_i = \sigma_j, \\ \left\{\frac{(\mathrm{prox}_{\lambda\|\cdot\|_\infty}(\sigma))_i - (\mathrm{prox}_{\lambda\|\cdot\|_\infty}(\sigma))_j}{\sigma_i - \sigma_j}\right\}, & \text{otherwise.} \end{cases} \tag{3.12}$$

For $p(\boldsymbol{X}) = \delta_{\mathbb{S}^n_+}(\boldsymbol{X})$, the corresponding generalized Jacobian operator can also be written as

$$D_{\mathbb{S}^n_+}(H) := Q\left(\Sigma \odot \left(Q^{\mathrm{T}}HQ\right)\right)Q^{\mathrm{T}}, \quad H \in \mathbb{S}^n, \tag{3.13}$$

where

$$\Sigma = \begin{bmatrix} E_{\alpha\alpha} & v_{\alpha\bar{\alpha}} \\ v^{\mathrm{T}}_{\alpha\bar{\alpha}} & 0 \end{bmatrix}, \quad v_{ij} := \frac{\lambda_i}{\lambda_i - \lambda_j}, \quad i \in \alpha, \quad j \in \bar{\alpha}, \tag{3.14}$$

where $E_{\alpha\alpha} \in \mathbb{S}^{|\alpha|}$ is the matrix of ones.

For given σ, denote $D^\tau = \frac{1}{\sigma}(I - D) + \tau I$, we next introduce a lemma that will be used to obtain $(D^\tau)^{-1}$ and preserve the low-rank structure. Since it can be verified directly, we omit the proof.

Lemma 3. *Let $\mathcal{T} : \mathbb{R}^{n\times n} \to \mathbb{R}^{n\times n} : \mathcal{T}(G) = \Omega_1 \odot G + \Omega_2 \odot G^{\mathrm{T}}$, then the inverse of $\mathcal{T}$ is*

$$\mathcal{T}^{-1}(G) = (\Omega_s + \Omega_a) \odot G + (\Omega_s - \Omega_a) \odot G^{\mathrm{T}},$$

where $\Omega_s = 1./[2(\Omega_1 + \Omega_2)]$, $\Omega_a = 1./[2(\Omega_1 - \Omega_2)]$ and ./ denotes elementwise division.

According to the above lemma, $(D^\tau)^{-1}$ can be represented as

$$(D^\tau)^{-1}(G) = U\ \left[(\Omega^\tau_s + \Omega^\tau_a) \odot G_1 + (\Omega^\tau_s - \Omega^\tau_a) \odot G_1^{\mathrm{T}}, (1./\Omega^\tau_3 \odot G_2)\right]V^{\mathrm{T}} + \sigma/(1+\sigma\tau)G. \tag{3.15}$$

where $\Omega^\tau_s = 1./[2(\Omega^\tau_1 + \Omega^\tau_2)] - \sigma/(1+\sigma\tau)E$, $\Omega^\tau_a = 1./[2(\Omega^\tau_1 - \Omega^\tau_2)]$, E is the matrix of ones with the correct size. The details of the computational process are summarized in Algorithm 0, from which the low-rank structure can be exploited effectively, and the total computational cost for each inner iteration reduces to $O(nr^2)$.

Algorithm 2 The process of computing $(D^\tau)^{-1}(G)$.

Require: $G, (\Omega_1^\tau)_{\alpha\alpha}, (\Omega_1^\tau)_{\alpha\bar{\alpha}}, (\Omega_2^\tau)_{\alpha\alpha}, (\Omega_2^\tau)_{\alpha\bar{\alpha}}, (\Omega_1^\tau)_{\alpha\beta}, U = [U_\alpha, U_{\bar{\alpha}}], V = [V_\alpha, V_{\bar{\alpha}}, V_\beta]$, where $U_\alpha \in \mathbb{R}^{n_1\times r}, U_{\bar{\alpha}} \in \mathbb{R}^{n\times(n_1-r)}, V_\alpha \in \mathbb{R}^{n_2\times r}, V_{\bar{\alpha}} \in \mathbb{R}^{n_2\times(n_2-r)}, V_\beta \in \mathbb{R}^{n_2\times(n_2-n_1)}$, penalty parameter σ and regularizer parameter τ.

Ensure: $(D^\tau)^{-1}(G)$

1: Compute $(G_1)_{\alpha\alpha}, (G_1)_{\alpha\bar{\alpha}}, (G_1)_{\bar{\alpha}\alpha}, (G_1)_{\alpha\beta}$ where

$$\begin{aligned}(G_1)_{\alpha\alpha} &= U_\alpha^{\mathrm{T}}\ G\ (V_\alpha), \qquad & (G_1)_{\alpha\bar{\alpha}} &= U_\alpha^{\mathrm{T}}\ G\ (V_{\bar{\alpha}}),\\ (G_1)_{\bar{\alpha}\alpha} &= U_{\bar{\alpha}}^{\mathrm{T}}\ G\ (V_{\bar{\alpha}}), \qquad & (G_1)_{\alpha\beta} &= U_\alpha^{\mathrm{T}}\ G\ (V_\beta).\end{aligned}$$

2: Compute

$$G_2 = \begin{bmatrix}(G_2)_{\alpha\alpha} & (G_2)_{\alpha\bar{\alpha}} & (G_2)_{\alpha\beta}\\ (G_2)_{\bar{\alpha}\alpha} & 0 & 0\end{bmatrix},$$

where

$$\begin{aligned}(G_2)_{\alpha\alpha} &= (\Omega_1^\tau)_{\alpha\alpha} \odot (G_1)_{\alpha\alpha} + (\Omega_2^\tau)_{\alpha\alpha} \odot ((G_1)_{\alpha\alpha})^{\mathrm{T}},\\ (G_2)_{\alpha\bar{\alpha}} &= (\Omega_1^\tau)_{\alpha\bar{\alpha}} \odot (G_1)_{\alpha\bar{\alpha}} + (\Omega_2^\tau)_{\alpha\bar{\alpha}} \odot ((G_1)_{\bar{\alpha}\alpha})^{\mathrm{T}},\\ (G_2)_{\bar{\alpha}\alpha} &= ((\Omega_1^\tau)_{\alpha\bar{\alpha}})^{\mathrm{T}} \odot (G_1)_{\bar{\alpha}\alpha} + ((\Omega_2^\tau)_{\alpha\bar{\alpha}})^{\mathrm{T}} \odot ((G_1)_{\alpha\bar{\alpha}})^{\mathrm{T}},\\ (G_2)_{\alpha\beta} &= (\Omega_1^\tau)_{\alpha\beta} \odot (G_1)_{\alpha\beta}.\end{aligned}$$

3: Compute $G_3 = G_{12} + G_{11} + G_{21} + G_{13}$ where

$$\begin{aligned}G_{11} &= U_\alpha\ (G_2)_{\alpha\alpha}\ V_\alpha^{\mathrm{T}}, \qquad & G_{12} &= U_\alpha\ (G_2)_{\alpha\bar{\alpha}}\ V_{\bar{\alpha}}^{\mathrm{T}},\\ G_{21} &= U_{\bar{\alpha}}\ (G_2)_{\bar{\alpha}\alpha}\ V_\alpha^{\mathrm{T}}, \qquad & G_{13} &= U_\alpha\ (G_2)_{\alpha\alpha}\ V_\beta^{\mathrm{T}}.\end{aligned}$$

4: Compute $(D^\tau)^{-1}(G) = G_3 + \sigma/(1+\sigma\tau)G$.

3.5 Fused regularizer

For the fused regularizer $p(x) = \lambda_1\|x\|_1 + \lambda_2\|Fx\|_1$, where $F(x) = [x_2 - x_1, \cdots, x_n - x_{n-1}]$, it follows from [26, Proposition 4] that the proximal operator of p is

$$\mathrm{prox}_p(\boldsymbol{v}) = \mathrm{prox}_{\lambda_1\|\cdot\|_1}(x_{\lambda_2}(\boldsymbol{v})) = \mathrm{prox}_{\lambda_1\|\cdot\|_1}(\boldsymbol{v} - F^{\mathrm{T}} z_{\lambda_2}(F\boldsymbol{v})), \tag{3.16}$$

where $z_{\lambda_2}(u) := \mathrm{argmin}_z \left\{ \frac{1}{2}\|F^T z\|^2 - \langle z, u\rangle \;\middle|\; \|z\|_\infty \le \lambda_2 \right\}, \forall u \in \mathbb{R}^{n-1}$. To characterize the generalized Jacobian of (3.16). we define the multifunction $\mathcal{M} : \mathbb{R}^n \to \mathbb{R}^{n\times n}$ as:

$$\mathcal{M}(v) := \{M \in \mathbb{R}^{n\times n} | M = \Theta P, \Theta \in \partial_B \mathrm{prox}_{\lambda_1\|\cdot\|_1}(x_{\lambda_2}), P \in \mathcal{P}_x(v)\}, \tag{3.17}$$

where $\mathcal{P}_x(v) := \{\hat{P} \in \mathbb{R}^{n-1\times n-1} | \hat{P} = I - F^{\mathrm{T}}(\Sigma_K F F^{\mathrm{T}} \Sigma_K)^\dagger F, K \in \mathcal{K}_z(v)\}$, $\Sigma_K = \mathrm{Diag}(\sigma_K) \in \mathbb{R}^{(n-1)\times(n-1)}$ and

$$(\sigma_K)_i = \begin{cases} 0, & \text{if } i \in K,\\ 1, & \text{otherwise}, i = 1, \cdots, n-1.\end{cases}$$

It follows from [26, Theorem 2] that $\mathcal{M}$ is nonempty and can be regarded as the generalized Jacobian of prox_p at $\boldsymbol{v}$. Furthermore, any element in $\mathcal{M}$ is symmetric and positive semidefinite. Let $\Gamma :=$

$I_n - F^{\mathrm{T}}(\Sigma F F^{\mathrm{T}}\Sigma)^{\dagger}F = \mathrm{Diag}(\Gamma_1, \cdots, \Gamma_N)$, where

$$\Gamma_i = \begin{cases} \frac{1}{n_i+1}\mathbf{E}_{n_i+1}, & \text{if } i \in J, \\ I_{n_i}, & \text{if } i \in \{1, N\}, \\ I_{n_i-1}, & \text{otherwise.} \end{cases}$$

It follows that $\Gamma = H + UU^{\mathrm{T}} = H + U_J U_J^{\mathrm{T}}$, where $H \in \mathbb{R}^{n\times n}$ is an N-block diagonal matrix given by $H = \mathrm{Diag}(\Upsilon_1, \ldots, \Upsilon_N)$ with

$$\Upsilon_i = \begin{cases} O_{n_i+1}, & \text{if } i \in J, \\ I_{n_i}, & \text{if } i \notin J \text{ and } i \in \{1, N\}, \\ I_{n_i-1}, & \text{otherwise.} \end{cases}$$

Furthermore, the (k, j)-th entry of $U \in \mathbb{R}^{n\times N}$ is given by

$$U_{k,j} = \begin{cases} \dfrac{1}{\sqrt{n_j+1}}, & \text{if } \displaystyle\sum_{t=1}^{j-1} n_t + 1 \le k \le \sum_{t=1}^{j} n_t + 1, \quad \text{and } j \in J, \\ 0, & \text{otherwise,} \end{cases} \tag{3.18}$$

and U_J consists of the nonzero columns of U, i.e., the columns indexed by J. Then $D = \Theta P \in \mathcal{M}$, where $P = I - F^{\mathrm{T}}(\Sigma F F^{\mathrm{T}}\Sigma)^{\dagger}F$, and

$$\theta_i = \begin{cases} 0, & \text{if } |(x_{\lambda_2}(v))_i| \le \lambda_1, \\ 1, & \text{otherwise,} \quad i = 1, \cdots, n. \end{cases}$$

Let $I_z(v) := \{i | |(z_{\lambda_2}(Bv))_i| = \lambda_2, i = 1, \cdots, n-1\}$, then $\Sigma = \mathrm{Diag}(\sigma) \in \mathbb{R}^{(n-1)\times(n-1)}$ with

$$\sigma_i = \begin{cases} 0, & \text{if } i \in I_z(v), \\ 1, & \text{otherwise}, i = 1, \cdots, n-1. \end{cases}$$

It follows that $\Theta \in \partial_B \mathrm{prox}_{\lambda_1\|\cdot\|_1}(x_{\lambda_2}(v))$ and $P \in \mathcal{P}_x(v)$. Therefore, we have $M = \Theta(H + U_J U_J^{\mathrm{T}}) = \Theta(H + U_J U_J^{\mathrm{T}})\Theta$, $\Theta^2 = \Theta$, $H^2 = H$, $\Theta H = \Theta H \Theta$. Define the index sets $\alpha_1 := \{i | \theta_i = 1, i \in \{1, \cdots, n\}\}$, $\quad \alpha_2 := \{i \,|h_i = 1, i \in \alpha_1\}$, where θ_i and h_i are the i-th diagonal entries of matrices Θ and H respectively. It then follows that

$$\mathcal{B}\Theta H \mathcal{B}^{\mathrm{T}} = \mathcal{B}\Theta H \Theta \mathcal{B}^{\mathrm{T}} = \mathcal{B}_{\alpha_1} H \mathcal{B}_{\alpha_1}^{\mathrm{T}} = \mathcal{B}_{\alpha_2}\mathcal{B}_{\alpha_2}^{\mathrm{T}},$$

where $\mathcal{B}_{\alpha_1} \in \mathbb{R}^{m\times|\alpha_1|}$ and $\mathcal{B}_{\alpha_2} \in \mathbb{R}^{m\times|\alpha_2|}$ are two submatrices obtained from $\mathcal{B}$ by extracting those columns with indices in α_1 and α_2. Meanwhile, we hvae

$$\mathcal{B}\Theta(U_j U_j^{\mathrm{T}})\mathcal{B}^{\mathrm{T}} = \mathcal{B}\Theta(U_j U_j^{\mathrm{T}})\Theta\mathcal{B}^{\mathrm{T}} = \mathcal{B}_{\alpha_1}\tilde{U}\tilde{U}^{\mathrm{T}}\mathcal{B}_{\alpha_1}^{\mathrm{T}},$$

where $\tilde{U} \in \mathbb{R}^{|\alpha_1|\times r}$ is a submatrix obtained from ΘU_J by extracting those rows with indices in α_1 and the zero columns in ΘU_J are removed. Therefore, by exploiting the structure in D, $\mathcal{B}D\mathcal{B}^{\mathrm{T}}$ can be expressed in the following form:

$$\mathcal{B}D\mathcal{B}^{\mathrm{T}} = \mathcal{B}_{\alpha_2}\mathcal{B}_{\alpha_2}^{\mathrm{T}} + \mathcal{B}_{\alpha_1}\tilde{U}\tilde{U}^{\mathrm{T}}\mathcal{B}_{\alpha_1}^{\mathrm{T}}.$$

For given σ, $D(D^{\tau})^{-1}D$, where $D^{\tau} = \frac{1}{\sigma}(I - D) + \tau I$ and $D = \Theta H \Theta$, we note that $D = \Theta H \Theta = \Theta H$ holds since $\Theta = \mathrm{Diag}(\Theta_1, \cdots, \Theta_N)$. It yields that $M = \mathrm{Diag}(\Theta_1\Gamma_1, \cdots, \Theta_N\Gamma_N)$. Define $J := \{j | \,\Gamma_j \text{ is not an identity matrix}, 1 \le j \le N\}$. It follows from $\mathrm{supp}(Fx_{\lambda_2}(v)) \subset K$ that $\Theta_j = \mathbf{O}_{n_j+1}$ or $I_{n_j+1}, \forall j \in$

J, which implies $\Theta_j\Gamma_j \in \mathbb{S}_+^{n_j+1}, \forall j \in J$ and hence $D \in \mathbb{S}_+^n$. Consequently, we have $D = \mathrm{Diag}(D_1, \cdots, D_n)$, where

$$D_i = \begin{cases} \frac{1}{n_i+1}\mathbf{E}_{n_i+1}, & \text{if } i \in J \text{ and } \Theta_i = I_{n_i}, \\ I_{n_i}, & \text{if } i \notin J \text{ and } i \in \{i, N\}, \\ 0, & \text{if } \Theta_i = \mathbf{0}, \\ I_{n_i-1}, & \text{otherwise.} \end{cases}$$

According to the SMW formula, the inverse of $(D^\tau)^{-1}$ has the explicit solution:

$$\left((\frac{1}{\sigma}+\tau)I - \frac{1}{\sigma}D\right)^{-1} = \left((\frac{1}{\sigma}+\tau)I - \frac{1}{\sigma}\Theta(H + U_J U_J^{\mathrm{T}})\Theta\right)^{-1}$$

$$= \begin{cases} \frac{\sigma}{1+\tau\sigma}I_{n_i+1} + \frac{1}{\tau(1+\tau\sigma)(n_i+1)}\mathbf{E}_{n_1+1}, & \text{if } \Theta_i = 1, \\ \frac{1}{\tau}I_{n_i}, & \text{if if } i \notin J \text{ and } i \in \{i, N\}, \\ \frac{\sigma}{1+\sigma\tau}I_{n_i}, & \text{if } \Theta_i = \mathbf{0}, \\ \frac{1}{\tau}I_{n_i-1}, & \text{otherwise.} \end{cases}$$

Consequently, $\overline{D} = \sigma D + D(D^\tau)^{-1}D$ can be represented by:

$$\overline{D} = \begin{cases} (\frac{1}{\tau}+\sigma)\frac{1}{n_i+1}\mathbf{E}_{n_1+1}, & \text{if } i \in J \text{ and } \Theta_i = I_{n_i}, \\ (\frac{1}{\tau}+\sigma)I_{n_i}, & \text{if if } i \notin J \text{ and } i \in \{i, N\}, \text{and } \Theta_i = I_{n_i}, \\ 0, & \text{if } \Theta_i = \mathbf{0}, \\ (\frac{1}{\tau}+\sigma)I_{n_i-1}, & \text{otherwise,} \end{cases}$$

and $D(D^\tau)^{-1}$ is:

$$D(D^\tau)^{-1} = \begin{cases} \frac{1}{\tau(n_i+1)}\mathbf{E}_{n_1+1}, & \text{if } i \in J \text{ and } \Theta_i = I_{n_i}, \\ \frac{1}{\tau}I_{n_i}, & \text{if if } i \notin J\,, i \in \{i, N\}, \text{and } \Theta_i = I_{n_i}, \\ 0, & \text{if } \Theta_i = \mathbf{0}, \\ \frac{1}{\tau}I_{n_i-1}, & \text{otherwise.} \end{cases}$$

Note that $\overline{D} = \Theta\tilde{H}$ and hence we have $\mathcal{B}\Theta\tilde{H}\mathcal{B}^{\mathrm{T}} = \mathcal{B}\Theta\tilde{H}\Theta\mathcal{B}^{\mathrm{T}} = \mathcal{B}_{\alpha_1}\widetilde{U}\widetilde{U}\mathcal{B}_{\alpha_1}^{\mathrm{T}} + \widetilde{\mathcal{B}}_{\alpha_2}\widetilde{\mathcal{B}}_{\alpha_2}^{\mathrm{T}}$, where $\widetilde{U}$ and $\widetilde{\mathcal{B}}_{\alpha_2}$ are the scaling matrices of U and $\mathcal{B}_{\alpha_2}$. This yields the decomposition: $\mathcal{B}\overline{D}\mathcal{B}^{\mathrm{T}} = W_1 W_2^{\mathrm{T}}$, where $W_1 := [\widetilde{\mathcal{B}}_{\alpha_2}, \mathcal{B}_{\alpha_1}\widetilde{U}\widetilde{U}^{\mathrm{T}}] \in \mathbb{R}^{m\times(|\alpha_1|+|\alpha_2|)}, W_2 = [\widetilde{\mathcal{B}}_{\alpha_2}, \mathcal{B}_{\alpha_1}]$. Using the above decomposition, we obtain

$$((\tau_1+1)I + \mathcal{B}\overline{D}\mathcal{B}^{\mathrm{T}})^{-1} = \frac{1}{\tau_1+1}I_m - \frac{1}{\tau_1+1}W_1((\tau_1+1)I_{|\alpha_1|+|\alpha_2|} + W_2^{\mathrm{T}}W_1)^{-1}W_2^{\mathrm{T}}.$$

Hence, we only need to factorize an $(|\alpha_1|+|\alpha_2|)\times(|\alpha_1|+|\alpha_2|)$ matrix and the total computational cost is merely $\mathcal{O}(|\alpha_1|+|\alpha_2|)^3 + \mathcal{O}(m(|\alpha_1|+|\alpha_2|)^2)$, matching the result in [26]. Consequently, we can solve the linear system using direct methods such as Cholesky factorization at low cost.

4 Numerical experiments

In this section, we conduct numerous experiments on different kinds of problems to verify the efficiency and robustness of Algorithm 1. The criteria to measure the accuracy are based on the KKT optimality conditions:

$$\eta = \max\{\eta_P, \eta_D, \eta_K, \eta_{\mathcal{P}}\},$$

where

$$\eta_P := \frac{\|\mathcal{A}(\boldsymbol{x}) - \Pi_{\mathcal{P}_2}(\mathcal{A}(\boldsymbol{x}) - \boldsymbol{y})\|}{1 + \|\boldsymbol{x}\|}, \eta_D := \frac{\|\mathcal{A}^*(\boldsymbol{y}) + \mathcal{B}^*(\boldsymbol{z}) + \boldsymbol{s} - \mathcal{Q}(\boldsymbol{v}) - \boldsymbol{c}\|}{1 + \|\boldsymbol{c}\|},$$

$$\eta_K := \min\left\{\frac{\|\boldsymbol{x} - \mathrm{prox}_p(\boldsymbol{x} - \boldsymbol{s})\|}{1 + \|\boldsymbol{s}\| + \|\boldsymbol{x}\|}, \frac{\|\mathcal{Q}(\boldsymbol{v}) - \mathcal{Q}(\boldsymbol{x})\|_{\mathrm{F}}}{1 + \|\mathcal{Q}(\boldsymbol{v})\| + \|\mathcal{Q}(\boldsymbol{x})\|}\right\},$$

$$\eta_{\mathcal{P}} := \min\left\{\frac{\|\mathrm{prox}_f(\mathcal{B}\boldsymbol{x} - \boldsymbol{z}) - \mathcal{B}(\boldsymbol{x})\|}{1 + \|\mathcal{B}(\boldsymbol{x})\| + \|\boldsymbol{z}\|} \text{ or } \frac{\|-\nabla f^*(-\boldsymbol{z}) - \mathcal{B}(\boldsymbol{x})\|}{1 + \|\mathcal{B}(\boldsymbol{x})\| + \|\boldsymbol{z}\|}, \frac{\|\Pi_{\mathcal{P}_1}(\boldsymbol{x} - \boldsymbol{r}) - \boldsymbol{x}\|}{1 + \|\boldsymbol{x}\| + \|\boldsymbol{r}\|}\right\}.$$

Denote `pobj` and `dobj` as the primal and dual objective function value. We also compute the relative gap by

$$\eta_g = \frac{\texttt{|pobj - dobj|}}{1 + \texttt{|pobj| + |dobj|}}.$$

Our software is available at `https://github.com/optsuite/SSNCVX`. All the experiments are done on a Linux server with a sixteen-core Intel Xeon Gold 6326 CPU and 256G memory.

4.1 Lasso

The Lasso problem corresponding to (1.1) can be expressed as

$$\min_{\boldsymbol{x}} \quad \frac{1}{2}\|\mathcal{B}(\boldsymbol{x}) - \boldsymbol{b}\|^2 + \lambda\|\boldsymbol{x}\|_1. \tag{4.1}$$

We test the problem on data from UCI[1] and LIBSVM dataset[2]. These datasets are collected from the 10-K Corpus [23] and the UCI data repository [29]. As suggested in [21], for the datasets **pyrim**, **triazines**, **abalone**, **bodyfat**, **housing**, **mpg**, and **space_ga**, we expand their original features by using polynomial basis functions over those features [25]. For example, the last digit in **pyrim5** indicates that an order 5 polynomial is used to generate the basis functions. This naming convention is also used in the rest of the expanded data sets. These numerical instances, shown in Table 2, can be quite difficult in terms of the dimensions and the largest eigenvalue of $\mathcal{B}\mathcal{B}^*$, which is denoted as $\lambda_{\max}(\mathcal{B}\mathcal{B}^*)$.

In Table 3, m denotes the number of samples, n denotes the number of features, and "nnz" denotes the number of nonzeros in the solution x using the following estimation:

$$\mathrm{nnz} := \min\{k | \sum_{i=1}^{k} |\hat{x}_i| \geq 0.999\|x\|_1\},$$

where $\hat{x}$ is obtained by sorting x such that $|\hat{x}_1| \geq |\hat{x}_2| \geq \cdots \geq |\hat{x}_n|$. The algorithms to compare are SSNAL[3], SLEP [30], and the ADMM algorithm. The numerical results for different choice of λ, i.e., $\lambda = 10^{-3}\|\mathcal{B}^{\mathrm{T}}\boldsymbol{b}\|_\infty$ and $\lambda = 10^{-4}\|\mathcal{B}^{\mathrm{T}}\boldsymbol{b}\|_\infty$ and different algorithms are given in Tables 3 and 4, where "nnz" denotes the number of nonzeros in the solution. We can see that both SSNCVX and SSNAL have successfully solved all problems, while other first-order methods can not. Furthermore, SSNCVX is competitive with SSNAL in all the tested Lasso problems, demonstrating its superiority in solving Lasso problems. For example, for the instance **log1p.E2006.train**, SSNCVX is twice as fast as SSNAL, while under the maximum time limit, SLEP and ADMM only achieve accuracies of 2.0e-2 and 1.2e-1, respectively.

[1] `https://archive.ics.uci.edu/`
[2] `https://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/`
[3] `https://github.com/MatOpt/SuiteLasso`

Probname	(m, n)	$\lambda_{\max}(\mathcal{B}\mathcal{B}^*)$
E2006.train	(3308, 72812)	1.912e+05
log1p.E2006.train	(16087,4265669)	5.86e+07
E2006.test	(3308,72812)	4.79e+04
log1p.E2006.test	(3308,1771946)	1.46e+07
pyrim5	(74,169911)	1.22e+06
triazines4	(186,557845)	2.07e+07
abalone7	(4177,6435)	5.21e+05
bodyfat7	(252,116280)	5.29e+04
housing7	(506,77520)	3.28e+05
mpg7	(392,3432)	1.28e+04
spacega9	(3107,5005)	4.01e+03

Table 2: Statistics of the UCI test instances.

id	nnz	SSNCVX		SSNAL		SLEP		ADMM	
		η	time	η	time	η	time	η	time
uci_CT	13	7.6e-7	**0.64**	4.4e-13	0.86	2.2e-2	35.95	7.7e-3	46.02
log1p.E2006.train	5	5.4e-7	**17.3**	1.5e-11	36.0	2.0e-2	1850.15	1.2e-1	3604.34
E2006.test	1	2.2e-11	**0.17**	4.3e-10	0.28	7.5e-12	1.11	7.9e-7	428.64
log1p.E2006.test	8	3.3e-8	**2.83**	2.5e-10	5.12	4.8e-2	447.56	1.2e-1	3603.64
pyrim5	72	4.2e-16	**1.82**	5.7e-8	2.16	2.4e-2	106.09	1.5e-3	3600.52
triazines4	519	2.6e-13	**10.64**	3.4e-9	11.23	8.3e-2	246.11	9.7e-3	3603.99
abalone7	24	4.6e-11	**0.75**	1.8e-9	1.06	2.5e-3	34.57	3.7e-4	540.27
bodyfat7	2	4.8e-13	**0.79**	1.4e-8	1.08	1.9e-6	28.10	8.4e-4	3609.63
housing7	158	5.1e-13	**1.83**	6.3e-9	1.74	1.3e-2	46.60	1.1e-2	3601.26
mpg7	47	4.4e-16	**0.10**	1.5e-8	0.14	7.4e-5	0.69	1.0e-6	63.41
spacega9	14	4.7e-15	**0.25**	9.7e-9	1.01	1.9e-8	21.12	1.0e-6	294.52
E2006.train	1	3.9e-9	**0.44**	4.4e-10	0.87	1.4e-11	1.13	4.4e-5	1149.22

Table 3: The results on Lasso problem ($\lambda = 10^{-3}\|\mathcal{B}^{\mathrm{T}}\boldsymbol{b}\|_\infty$).

4.2 Fused Lasso

The Fused Lasso problem corresponding to (1.1) can be expressed as

$$\min_{\boldsymbol{x}} \quad \frac{1}{2}\|\mathcal{B}(\boldsymbol{x}) - \boldsymbol{b}\|^2 + \lambda_1\|\boldsymbol{x}\|_1 + \lambda_2\|F\boldsymbol{x}\|. \tag{4.2}$$

We compare SSNCVX with SSNAL [26], ADMM, and SLEP [30] solvers. Consistent with the Lasso problem, we also test the problems with data from the UCI data and the LIBSVM dataset. The numerical experiments for UCI datasets are listed in Table 5. It is shown that SSNCVX has comparable performance to SSNAL and better performance than ADMM and SLEP.

4.3 QP

The QP problem is also a special case of (1.1). In this subsection, we consider solving portfolio optimization, an application of QP and is widely used in the investment community:

$$\min_{\boldsymbol{x}} \langle \boldsymbol{x}, \mathcal{Q}(\boldsymbol{x})\rangle + \langle \boldsymbol{c}, \boldsymbol{x}\rangle, \quad \text{s.t.} \quad \langle \boldsymbol{e}_n, \boldsymbol{x}\rangle = 1, \quad \boldsymbol{x} \geq \boldsymbol{0}, \tag{4.3}$$

id	nnz	SSNCVX		SSNAL		SLEP		ADMM	
		η	time	η	time	η	time	η	time
uci_CT	44	2.6e-7	**1.26**	2.9e-12	1.75	1.8e-1	41.63	2.0e-3	49.88
log1p.E2006.train	599	3.0e-7	**33.92**	5.9e-11	68.83	3.3e-2	1835.32	1.2e-1	3608.17
E2006.test	1	2.6e-14	**0.20**	3.7e-9	0.29	2.4e-12	0.38	9.0e-7	268.11
log1p.E2006.test	1081	8.8e-9	**13.72**	2.7e-10	30.1	7.5e-2	455.56	1.6e-1	3606.60
pyrim5	78	5.6e-16	**2.01**	5.0e-7	2.59	1.1e-2	108.93	3.1e-3	3601.09
triazines4	260	9.5e-16	**18.48**	8.3e-8	34.44	9.2e-2	187.45	1.2e-2	3604.48
abalone7	59	6.1e-12	**1.63**	1.2e-8	2.00	1.5e-2	43.91	1.0e-6	356.34
bodyfat7	3	1.0e-16	**1.14**	9.7e-8	1.51	6.1e-4	41.98	1.3e-4	3601.89
housing7	281	2.6e-11	**2.51**	1.2e-7	2.52	4.1e-2	52.60	3.6e-4	3601.09
mpg7	128	1.8e-15	**0.11**	6.9e-8	0.18	5.8e-4	0.76	9.9e-7	11.67
spacega9	38	3.1e-12	**0.53**	3.5e-7	0.72	9.0e-5	22.96	1.0e-6	53.23
E2006.train	1	5.6e-9	**0.75**	4.4e-9	0.88	1.0e-11	1.39	4.4e-5	1132.34

Table 4: The results of tested algorithms on Lasso problem ($\lambda = 10^{-4}\|\mathcal{B}^{\mathrm{T}}\boldsymbol{b}\|_\infty$).

id	nnz(x)	nnz(Bx)	SSNCVX		SSNAL		SLEP		ADMM	
			η	time	η	time	η	time	η	time
uci_CT	8	1	6.3e-7	**0.25**	7.9e-7	0.42	1.8e-6	2.06	7.7e-3	41.75
log1p.E2006.train	31	2	2.8e-7	**10.43**	2.4e-7	14.02	1.2e-2	4889.15	1.2e-1	3623.18
E2006.test	1	1	1.5e-7	**0.17**	5.1e-7	0.33	4.8e-8	0.93	8.2e-7	1768.26
log1p.E2006.test	33	1	4.1e-7	**2.60**	8.1e-7	2.74	1.2e-2	1690.60	2.4e-2	3601.25
pyrim5	1135	74	9.1e-7	**2.34**	4.5e-7	3.40	3.4e-2	238.43	2.4e-3	3601.20
triazines4	2666	206	2.1e-7	**10.24**	9.8e-7	15.49	7.8e-2	585.70	2.8e-2	3601.89
bodyfat7	63	8	3.0e-7	**0.72**	7.2e-9	1.35	9.9e-7	41.13	3.5e-3	3612.99
abalone7	1	1	1.6e-7	**0.83**	5.3e-8	0.95	1.3e-3	32.51	6.4e-4	538.90
housing7	205	47	7.6e-7	**1.98**	8.2e-7	2.73	5.0e-3	117.07	2.2e-2	3600.28
mpg7	42	20	1.9e-7	**0.08**	1.8e-7	0.11	3.4e-6	3.19	6.3e-6	156.31
spacega9	24	11	5.0e-8	**0.27**	1.2e-7	0.44	6.1e-8	5.32	9.9e-7	337.14
E2006.train	1	1	3.7e-7	**0.42**	4.0e-8	0.98	9.7e-12	0.39	4.3e-5	1196.42

Table 5: The results of tested algorithms on Fused Lasso problem ($\lambda_1 = 10^{-3}\|\mathcal{B}^* b\|_\infty$ and $\lambda_2 = 5\lambda_1$.)

where $\boldsymbol{x}$ denotes the decision variable, $\mathcal{Q} \in \mathcal{S}^n_+$ denotes the data matrix, $\gamma > 0$, and $\boldsymbol{e}_n$ is the vector of ones. The $\mathcal{Q}$ and $\boldsymbol{c}$ are chosen from Maros-Mészáros dataset [10] and synthetic data. For Maros-Mészáros dataset, we choose the problem whose dimension is more than 10000 since the data is highly sparse. For synthetic data, we generate our test data randomly via the following Matlab script as follows [28]:

```
p = 0.01*n;
F = sprandn(n, p, 0.1); D = sparse(diag(sqrt(p)*rand(n,1)));
Q = cov(F') + D;
c = randn(n,1);
```

where `n` denots the dimision. We compare SSNCVX with the HIGHS [22] solver. The results are listed in Table 7. It is shown that SSNCVX can solve all the tested problems while HiGHS can not.

id	nnz(x)	nnz(Bx)	SSNCVX		SSNAL		SLEP		ADMM	
			η	time	η	time	η	time	η	time
uci_CT	18	8	6.3e-7	**0.40**	8.9e-10	0.42	1.8e-6	2.06	7.7e-3	39.29
log1p.E2006.train	8	3	7.0e-7	**8.37**	1.5e-7	12.6	1.2e-2	4889.15	1.2e-1	3606.14
E2006.test	1	1	1.5e-7	**0.17**	2.9e-8	0.33	4.8e-8	0.93	7.7e-7	699.27
log1p.E2006.test	32	5	3.1e-9	**3.07**	1.2e-8	3.31	1.2e-2	1690.60	7.9e-2	3601.20
pyrim5	327	97	9.1e-7	**2.34**	2.0e-7	3.06	3.4e-2	238.43	1.5e-3	3601.13
triazines4	1244	286	8.2e-7	**10.51**	2.4e-7	12.63	7.8e-2	585.70	2.8e-2	3603.56
bodyfat7	2	3	2.8e-8	**0.81**	4.7e-8	0.89	9.9e-7	41.13	2.7e-3	3606.85
abalone7	26	15	3.7e-7	**0.49**	5.0e-9	1.17	1.3e-3	32.51	5.0e-4	545.23
housing7	131	117	6.4e-7	**1.46**	3.9e-7	2.4	5.0e-3	117.07	2.0e-2	3603.08
mpg7	32	39	6.7e-7	**0.07**	2.2e-7	0.15	3.4e-6	3.19	1.0e-6	77.58
spacega9	14	13	8.7e-7	**0.22**	1.7e-7	0.44	6.1e-8	5.32	1.0e-6	333.39
E2006.train	1	1	4.2e-7	**0.45**	4.0e-7	1.12	9.7e-12	0.39	4.4e-5	1189.36

Table 6: The results of tested algorithms on Fused Lasso problem ($\lambda_1 = 10^{-3}\|\mathcal{B}^* b\|_\infty$ and $\lambda_2 = \lambda_1$.)

	SSNCVX			HIGHS		
problem	obj	η	time	obj	η	time
Aug2D	-1.0e+0	2.9e-11	**0.25**	-	-	-
Aug2DC	-1.0e+0	7.5e-13	**0.20**	-	-	-
Aug2DCQP	-1.0e+0	7.5e-13	**0.18**	-	-	-
Aug2DQP	-1.0e+0	1.7e-16	**0.31**	-	-	-
BOYD1	-1.1e+4	2.0e-7	**47.80**	-	-	-
BOYD2	-1.0e+1	2.3e-9	**0.29**	-1.0e+1	4.3e-6	3667.91
CONT-100	-3.3e-4	7.0e-12	**1.23**	-3.3e-4	7.8e-4	122.04
CONT-101	-9.9e-5	0.0e+0	**0.07**	-9.9e-5	4.5e-3	3600.04
CONT-200	-8.3e-5	4.3e-8	**3.96**	-8.3e-5	3.2e-3	3600.09
CONT-201	-2.5e-5	0.0e+0	**0.16**	-	-	-
CONT-300	-1.1e-5	0.0e+0	**0.24**	-1.1e-5	0.0e+0	4011.90
DTOC-3	1.3e-8	8.9e-18	**0.39**	-	-	-
LISWET1	-1.1e+0	2.3e-18	**0.15**	-1.1e+0	6.8e-6	0.70
UBH1	-0.0e+0	4.8e-9	**0.28**	-	-	-
random512_1	-2.6e+0	7.2e-11	**0.36**	-2.6e+0	2.1e-7	1.10
random512_2	-2.2e+0	7.4e-13	**0.40**	-2.2e+0	2.5e-7	1.11
random1024_1	-2.3e+0	2.2e-9	**1.41**	-2.3e+0	4.0e-7	2.32
random1024_2	-2.5e+0	2.7e-8	**0.81**	-2.5e+0	2.5e-7	2.32
random2048_1	-2.6e+0	1.7e-7	**3.40**	-2.6e+0	2.5e-7	3.96
random2048_2	-2.2e+0	2.6e-10	**2.92**	-2.2e+0	1.4e-7	4.06

Table 7: Computational results of tested algorihtms on portfolio optimization.

4.4 SOCP

The SOCP problem corresponding to (1.1) is formulated as:

$$\min_{\boldsymbol{x}} \langle \boldsymbol{c}, \boldsymbol{x} \rangle \quad \text{s.t.}\, \mathcal{A}(\boldsymbol{x}) = \boldsymbol{b}, \quad \boldsymbol{x} \in \mathcal{Q}^n, \tag{4.4}$$

where $\mathcal{Q}^n = \mathcal{Q}_1 \times \mathcal{Q}_2 \times \cdots \times \mathcal{Q}_n$ and $\mathcal{Q}_i = \{(x_0, \bar{x}) \in \mathbb{R}^{n_i} | x_0 \geq \|\bar{x}\|_2)\}$ represents second-order cone. For the SOCP case, we test the CBLIB problems [17] listed in Hans Mittelmann's SOCP Benchmark [33]. Table 8 compares the running time of SSNCVX with the commonly used solvers ECOS [16], SDPT3 [42], and MOSEK [2] under a 3600-second time limit.

Note that the MATLAB solvers (SSNCVX and SDPT3) solve the preprocessed datasets with preprocessing time excluded. This preprocessing, which typically requires several seconds, significantly reduced solution times for some instances (e.g., firL2a), making these solvers appear faster for such problems. However, as geometric means are calculated with a 10-second shift, the exclusion has a negligible impact on the overall results. On these problems, SSNCVX is 70% faster than SDPT3, though both remain slower than commercial solver MOSEK. Compared with SDPT3, SSNCVX also exhibits the additional advantage of handling sparse and dense columns separately. Notably, SSNCVX can solve problems like beam7 if we don't set a time limit, while SDPT3 fails due to out of memory.

id	SSNCVX		SDPT3		ECOS		MOSEK	
	η	time	η	time	η	time	η	time
beam7	-	-	-	-	1.0e-7	206.0	6.0e-4	19.7
beam30	-	-	-	-	3.0e-7	2464.7	3.0e-6	96.5
chainsing-50000-1	1.5e-7	5.8	6.9e-7	5.5	-	-	1.6e-6	3.8
chainsing-50000-2	7.3e-7	14.4	7.0e-7	9.5	-	-	1.0e-7	4.1
chainsing-50000-3	5.0e-9	15.7	1.4e-7	19.4	-	-	1.0e-8	2.0
db-joint-soerensen	-	-	-	-	-	-	2.0e-8	36.3
db-plate-yield-line	8.5e-7	597.2	8.7e-7	217.6	-	-	5.0e-7	6.2
dsNRL	1.0e-6	859.2	8.9e-7	567.8	-	-	8.2e-10	67.1
firL1	5.3e-11	101.6	7.8e-7	582.0	3.0e-8	1305.2	3.1e-9	20.5
firL1Linfalph	8.4e-7	509.6	7.5e-7	916.2	3.0e-8	2846.6	4.0e-9	91.8
firL1Linfeps	7.0e-7	86.4	8.2e-7	179.1	2.0e-9	2530.8	3.0e-8	27.5
firL2a	1.4e-8	0.4	6.1e-7	0.1	2.0e-9	944.6	2.0e-13	4.4
firL2L1alph	1.1e-7	37.4	7.3e-7	131.7	3.0e-9	201.5	2.2e-10	5.8
firL2L1eps	2.0e-9	159.5	6.2e-7	586.0	2.0e-8	796.6	3.5e-9	17.2
firL2Linfalph	7.9e-7	89.1	7.9e-7	799.9	-	-	9.0e-9	41.7
firL2Linfeps	5.2e-7	72.4	8.0e-9	251.2	5.0e-10	687.1	1.0e-8	29.9
firLinf	1.4e-7	280.2	7.1e-7	576.7	5.0e-9	3478.7	1.0e-8	123.6
wbNRL	8.7e-7	20.1	5.9e-7	151.2	5.0e-9	1332.6	2.4e-9	11.8
geomean	-	155.0	-	267.8	-	1731.4	-	22.7

Table 8: The results on Hans Mittelmann's SOCP benchmark.

4.5 SPCA

The sparse PCA problem for a single component is

$$\max_{\boldsymbol{y}} \boldsymbol{y}^T \boldsymbol{L} \boldsymbol{y}, \quad \text{s.t.} \quad \|\boldsymbol{y}\|_2 = 1, \quad \text{card}(\boldsymbol{y}) \leq k.$$

The function card$(\cdot)$ refers to the number of nonzero elements. This problem can be expressed as a low-rank SDP:

$$\min_{\boldsymbol{X}} -\langle \boldsymbol{L}, \boldsymbol{X} \rangle + \lambda \|\boldsymbol{X}\|_1, \text{ s.t. } \text{Tr}(\boldsymbol{X}) = 1, \quad \boldsymbol{X} \succeq 0. \tag{4.5}$$

We formulate $\boldsymbol{L}$ based on the covariance matrix of real data or use the random example in [50]. For random examples, $\boldsymbol{L}$ is generated by: $\boldsymbol{L} = \frac{1}{\|\boldsymbol{u}\|_2}\boldsymbol{u}\boldsymbol{u}^T + VV^{\mathrm{T}}$, where $\boldsymbol{u} = [1, 1/2, \ldots, 1/n]$ and each entry of $V \in \mathbb{R}^{n\times n}$ is randomly uniformly chosen from $[0, 1]$. We compare SSNCVX with SuperSCS [37]. The maximum iteration time is set to 3600s. The results are presented in Table 9. Compared with SuperSCS, SSNCVX solves SPCA faster and achieves higher accuracy.

Here's the modified table with scientific notation using "e" in LaTeX code:

	SSNCVX			superSCS		
problem	obj	η	time	obj	η_K	time
20news	-3.3e+3	2.0e-12	0.8	-3.3e+3	1.0e-6	9.6
bibtex	-1.8e+4	1.2e-11	76.6	-1.7e+4	2.7e-1	3626.4
colon_cancer	-1.8e+4	5.5e-12	45.9	-1.4e+4	4.9e-1	3647.9
delicious	-7.5e+4	2.6e-12	2.9	-7.5e+4	2.5e-3	2813.5
dna	-1.8e+3	1.2e-13	0.3	-1.8e+3	1.0e-6	29.2
gisette	-3.9e+5	2.5e-12	1190.0	-1.3e+5	7.0e-1	3703.5
madelon	-9.5e+7	5.9e-15	16.7	-9.5e+7	4.4e-5	3343.6
mnist	-2.0e+10	4.0e-17	15.7	-2.0e+10	1.0e-6	195.4
protein	-3.0e+3	3.5e-11	3.7	-3.0e+3	8.7e-3	2334.1
random1024_1	-5.2e+5	9.3e-18	2.8	-5.3e+5	3.2e-2	3603.3
random1024_2	-5.2e+5	4.4e-18	2.7	-5.2e+5	1.9e-3	3604.8
random1024_3	-5.2e+5	1.3e-17	2.8	-5.2e+5	1.4e-3	3608.3
random2048_1	-2.1e+6	7.8e-18	3.3	-2.0e+6	2.3e-1	3605.5
random2048_2	-2.1e+6	5.1e-18	3.5	-2.1e+6	5.9e-2	3607.0
random2048_3	-2.1e+6	1.5e-18	2.3	-2.1e+6	1.5e-2	3608.2
random4096_1	-8.4e+6	8.2e-18	73.4	-1.0e+0	N/A	3655.4
random4096_2	-8.4e+6	3.5e-18	73.1	-8.3e+6	1.2e-2	3638.0
random4096_3	-8.4e+6	6.7e-19	72.4	-8.4e+6	9.6e-3	3645.0
random512_1	-1.3e+5	4.3e-18	0.6	-1.3e+5	1.0e-6	252.0
random512_2	-1.3e+5	1.1e-17	0.6	-1.3e+5	8.1e-3	2938.5
random512_3	-1.3e+5	5.7e-18	0.6	-1.3e+5	8.2e-3	2802.0
usps	-1.2e+5	2.4e-13	1.1	-1.2e+5	1.0e-6	229.8

Table 9: Computational results of SSNCVX and superSCS on SPCA.

4.6 LRMC

Low-rank matrix recovery (LRMC) is a classical problem in image processing [45]. The LRMC problem corresponding to (1.1) is represented by

$$\min_{\boldsymbol{X}} \|\mathcal{B}(\boldsymbol{X}) - \boldsymbol{B}\|_{\mathrm{F}}^2 + \lambda\|\boldsymbol{X}\|_*. \tag{4.6}$$

We compare SSNCVX with classical ADMM, proximal gradient, and accelerated proximal gradient method on 8 images. The tested images are listed in Figure 1. The tested images are corrupted by randomly choosing 50 percent of the pixels. The results are listed in Table 10. It is shown that SSNCVX not only has higher accuracy but also is the fastest compared with the tested first-order methods.

Figure 1: The eight tested images for LRMC.

Problem	SSNCVX		ADMM		PG		APG	
	η	Time	η	Time	η	Time	η	Time
Image1	1.5e-9	**20.1**	9.9e-9	84.5	9.6e-9	122.4	9.9e-9	55.8
Image2	4.3e-9	**22.1**	1.0e-8	84.0	9.8e-9	120.9	9.6e-9	54.5
Image3	5.3e-9	**23.2**	9.9e-9	82.8	9.6e-9	119.5	9.3e-9	53.9
Image4	3.3e-9	**25.3**	9.7e-9	84.1	9.8e-9	121.1	9.8e-9	54.6
Image5	7.4e-9	**20.3**	9.5e-9	83.7	9.7e-9	120.4	9.9e-9	54.4
Image6	1.9e-9	**20.9**	1.0e-8	83.5	9.8e-9	120.4	9.7e-9	54.3
Image7	1.6e-9	**20.2**	9.9e-9	82.2	9.9e-9	118.3	9.7e-9	53.1
Image8	2.3e-9	**20.8**	9.8e-9	83.0	9.7e-9	120.0	9.7e-9	53.9

Table 10: Comparison of tested algorithms on the LRMC problem.

5 Conclusion

In this paper, we propose SSNCVX, a semismooth Newton-based algorithmic framework for solving convex composite optimization problems. By reformulating the problem through augmented Lagrangian duality and characterizing the optimality condition via a semismooth equation system, our method provides a unified approach to handle multi-block problems with nonsmooth terms. The framework eliminates the need for problem-specific transformations while enabling flexible model modifications through simple interface updates. Featuring a single-loop structure with second-order semismooth Newton steps, SSNCVX demonstrates superior efficiency and robustness in extensive numerical experiments, outperforming state-of-the-art solvers across various applications. Numerical experiments on various problems establish SSNCVX as an effective and versatile tool for large-scale convex optimization.

References

[1] M. F. Anjos and J. B. Lasserre, *Handbook on semidefinite, conic and polynomial optimization*, vol. 166, Springer Science & Business Media, 2011.

[2] M. ApS, *The MOSEK optimization toolbox for MATLAB manual. Version 10.1.0.*, 2019, `http://docs.mosek.com/10.1/toolbox/index.html`.

[3] G. Bareilles, F. Iutzeler, and J. Malick, *Newton acceleration on manifolds identified by proximal gradient methods*, Mathematical Programming, 200 (2023), pp. 37–70.

[4] A. Beck, *First-order Methods in Optimization*, SIAM, 2017.

[5] A. Beck and N. Guttmann-Beck, *Fom–a matlab toolbox of first-order methods for solving convex optimization problems*, Optimization Methods and Software, 34 (2019), pp. 172–193.

[6] S. R. Becker, E. J. Candès, and M. C. Grant, *Templates for convex cone problems with applications to sparse signal recovery*, Mathematical programming computation, 3 (2011), pp. 165–218.

[7] A. Ben-Tal and A. Nemirovski, *Lectures on modern convex optimization: analysis, algorithms, and engineering applications*, SIAM, 2001.

[8] S. Boyd, N. Parikh, E. Chu, B. Peleato, J. Eckstein, et al., *Distributed optimization and statistical learning via the alternating direction method of multipliers*, Foundations and Trends® in Machine learning, 3 (2011), pp. 1–122.

[9] E. Candes and T. Tao, *The dantzig selector: Statistical estimation when p is much larger than n*, (2007).

[10] S. Caron, A. Zaki, P. Otta, D. Arnström, J. Carpentier, F. Yang, and P.-A. Leziart, *qpbenchmark: Benchmark for quadratic programming solvers available in Python*, 2025, `https://github.com/qpsolvers/qpbenchmark`.

[11] G. B. Dantzig, *Linear programming*, Operations research, 50 (2002), pp. 42–47.

[12] Q. Deng, Q. Feng, W. Gao, D. Ge, B. Jiang, Y. Jiang, J. Liu, T. Liu, C. Xue, Y. Ye, et al., *New developments of ADMM-based interior point methods for linear programming and conic programming*, arXiv preprint arXiv:2209.01793, (2022).

[13] Z. Deng, K. Deng, J. Hu, and Z. Wen, *An augmented lagrangian primal-dual semismooth newton method for multi-block composite optimization*, Journal of Scientific Computing, 102 (2025), p. 65.

[14] Z. Deng, J. Hu, K. Deng, and Z. Wen, *An efficient primal dual semismooth newton method for semidefinite programming*, arXiv preprint arXiv:2504.14333, (2025).

[15] S. Diamond and S. Boyd, *Cvxpy: A python-embedded modeling language for convex optimization*, Journal of Machine Learning Research, 17 (2016), pp. 1–5.

[16] A. Domahidi, E. Chu, and S. Boyd, *ECOS: An SOCP solver for embedded systems*, in 2013 European control conference (ECC), IEEE, 2013, pp. 3071–3076.

[17] H. A. Friberg, *Cblib 2014: a benchmark library for conic mixed-integer and continuous optimization*, Mathematical Programming Computation, 8 (2016), pp. 191–214.

[18] M. Grant, S. Boyd, and Y. Ye, *Cvx: Matlab software for disciplined convex programming*, 2008.

[19] J.-B. Hiriart-Urruty, J.-J. Strodiot, and V. H. Nguyen, *Generalized hessian matrix and second-order optimality conditions for problems with c 1, 1 data*, Applied mathematics and optimization, 11 (1984), pp. 43–56.

[20] J. Hu, T. Tian, S. Pan, and Z. Wen, *On the analysis of semismooth Newton-type methods for composite optimization*, Journal of Scientific Computing, 103 (2025), pp. 1–31.

[21] L. Huang, J. Jia, B. Yu, B.-G. Chun, P. Maniatis, and M. Naik, *Predicting execution time of computer programs using sparse polynomial regression*, Advances in neural information processing systems, 23 (2010).

[22] Q. Huangfu and J. J. Hall, *Parallelizing the dual revised simplex method*, Mathematical Programming Computation, 10 (2018), pp. 119–142.

[23] S. Kogan, D. Levin, B. R. Routledge, J. S. Sagi, and N. A. Smith, *Predicting risk from financial reports with regression*, in Proceedings of human language technologies: the 2009 annual conference of the North American Chapter of the Association for Computational Linguistics, 2009, pp. 272–280.

[24] A. S. Lewis, J. Liang, and T. Tian, *Partial smoothness and constant rank*, SIAM Journal on Optimization, 32 (2022), pp. 276–291.

[25] X. Li, D. Sun, and K.-C. Toh, *A highly efficient semismooth Newton augmented Lagrangian method for solving Lasso problems*, SIAM Journal on Optimization, 28 (2018), pp. 433–458.

[26] X. Li, D. Sun, and K.-C. Toh, *On efficiently solving the subproblems of a level-set method for fused Lasso problems*, SIAM Journal on Optimization, 28 (2018), pp. 1842–1866.

[27] Y. Li, Z. Wen, C. Yang, and Y.-x. Yuan, *A semismooth Newton method for semidefinite programs and its applications in electronic structure calculations*, SIAM Journal on Scientific Computing, 40 (2018), pp. A4131–A4157.

[28] L. Liang, X. Li, D. Sun, and K.-C. Toh, *Qppal: a two-phase proximal augmented lagrangian method for high-dimensional convex quadratic programming problems*, ACM Transactions on Mathematical Software (TOMS), 48 (2022), pp. 1–27.

[29] M. Lichman et al., *Uci machine learning repository*, 2013.

[30] J. Liu, S. Ji, J. Ye, et al., *Slep: Sparse learning with efficient projections*, Arizona State University, 6 (2009), p. 7.

[31] Y. Liu, Z. Wen, and W. Yin, *A multiscale semi-smooth Newton method for optimal transport*, Journal of Scientific Computing, 91 (2022), p. 39.

[32] R. Mifflin, *Semismooth and semiconvex functions in constrained optimization*, SIAM Journal on Control and Optimization, 15 (1977), pp. 959–972.

[33] H. D. Mittelmann, *An independent benchmarking of SDP and SOCP solvers*, Mathematical Programming, 95 (2003), pp. 407–430, `https://plato.asu.edu/ftp/socp.html`.

[34] B. O'Donoghue, *Operator splitting for a homogeneous embedding of the linear complementarity problem*, SIAM Journal on Optimization, 31 (2021), pp. 1999–2023.

[35] G. Optimization, *Gurobi optimizer reference manual, version 9.5*, Gurobi Optimization, (2021).

[36] B. O'donoghue, E. Chu, N. Parikh, and S. Boyd, *Conic optimization via operator splitting and homogeneous self-dual embedding*, Journal of Optimization Theory and Applications, 169 (2016), pp. 1042–1068.

[37] P. Sopasakis, K. Menounou, and P. Patrinos, *Superscs: fast and accurate large-scale conic optimization*, in 2019 18th European Control Conference (ECC), IEEE, 2019, pp. 1500–1505.

[38] J. F. Sturm, *Using SeDuMi 1.02, a MATLAB toolbox for optimization over symmetric cones*, Optimization methods and software, 11 (1999), pp. 625–653.

[39] D. Sun, K.-C. Toh, and L. Yang, *A convergent 3-block semiproximal alternating direction method of multipliers for conic programming with 4-type constraints*, SIAM Journal on Optimization, 25 (2015), pp. 882–915.

[40] D. Sun, K.-C. Toh, Y. Yuan, and X.-Y. Zhao, *SDPNAL+: A matlab software for semidefinite programming with bound constraints (version 1.0)*, Optimization Methods and Software, 35 (2020), pp. 87–115.

[41] A. Themelis, M. Ahookhosh, and P. Patrinos, *On the acceleration of forward-backward splitting via an inexact Newton method*, Splitting Algorithms, Modern Operator Theory, and Applications, (2019), pp. 363–412.

[42] K.-C. Toh, M. J. Todd, and R. H. Tütüncü, *SDPT3— A MATLAB software package for semidefinite programming, version 1.3*, Optimization methods and software, 11 (1999), pp. 545–581.

[43] Y. Wang, K. Deng, H. Liu, and Z. Wen, *A decomposition augmented Lagrangian method for low-rank semidefinite programming*, SIAM Journal on Optimization, 33 (2023), pp. 1361–1390.

[44] Z. Wen, D. Goldfarb, and W. Yin, *Alternating direction augmented Lagrangian methods for semidefinite programming*, Mathematical Programming Computation, 2 (2010), pp. 203–230.

[45] Z. Wen, W. Yin, and Y. Zhang, *Solving a low-rank factorization model for matrix completion by a nonlinear successive over-relaxation algorithm*, Mathematical Programming Computation, 4 (2012), pp. 333–361.

[46] H. Wolkowicz, R. Saigal, and L. Vandenberghe, *Handbook of semidefinite programming: theory, algorithms, and applications*, vol. 27, Springer Science & Business Media, 2012.

[47] X. Xiao, Y. Li, Z. Wen, and L. Zhang, *A regularized semi-smooth Newton method with projection steps for composite convex programs*, Journal of Scientific Computing, 76 (2018), pp. 364–389.

[48] L. Yang, D. Sun, and K.-C. Toh, *SDPNAL+: A majorized semismooth Newton-CG augmented Lagrangian method for semidefinite programming with nonnegative constraints*, Mathematical Programming Computation, 7 (2015), pp. 331–366.

[49] M.-C. Yue, Z. Zhou, and A. M.-C. So, *A family of inexact SQA methods for non-smooth convex minimization with provable convergence guarantees based on the Luo–Tseng error bound property*, Mathematical Programming, 174 (2019), pp. 327–358.

[50] Y. Zhang, A. d'Aspremont, and L. E. Ghaoui, *Sparse pca: Convex relaxations, algorithms and applications*, Handbook on Semidefinite, Conic and Polynomial Optimization, (2012), pp. 915–940.